

 **DTU Compute**
Department of Applied Mathematics and Computer Science

Artificial Intelligence and Human Cognition Lecture Notes

v. 1.25.3

©Tobias Andersen

These lecture notes are intended for use in teaching
at the Technical University of Denmark. Please do
not distribute or share outside their intended use.



Contents

Contents	i
1 Introduction to cognitive science and artificial intelligence	1
1.1 Human vs. artificial intelligence	1
1.2 Levels of computation	2
1.3 Learning	4
1.4 Human vs Computer Memory	5
1.5 Human working memory and long-term memory	6
1.6 Human working memory	7
1.7 AI mimicking human cognition	9
1.8 Mini project	10
2 Neural encoding	12
2.1 Linear encoding	12
2.2 Lateral inhibition	13
2.3 Principal Component Analysis (PCA) in linear encoding	18
3 Bayesian models of perception	22
3.1 Introduction	22
3.2 The use of prior information in visual perception	23
3.3 The use of marginalisation in perception	29
4 Human category learning	34
Bibliography	37

CHAPTER 1

Introduction to cognitive science and artificial intelligence

1.1 Human vs. artificial intelligence

A first step in comparing human and artificial intelligence is to determine whether the word ‘intelligence’ means the same in those two contexts. Intelligence generally refers to the ability to learn from experience and apply what is learned to new situations. In this sense, human and machine intelligence are similar but when we begin to break down learning into separate domains, we find that the domains of human and machine intelligence are quite different.

How can we test the intelligence of humans? One approach would be to test their ability to learn in a domain that we believe requires intelligence to master. We could, for example, devise a test in learning mathematical skills because we believe that intelligence is required to perform well in such tests. However, we could also devise a test of learning linguistic skills. If only one general form of intelligence is required to master both tests then we would expect performance on the two tests to correlate strongly. If the correlation is weak then our measure of intelligence will be dependent on which test we use, and we must conclude that there is not general intelligence, but rather a number of specialised intelligences. Whether intelligence is general across all domains or whether humans have multiple intelligences that are domain specific has been a fundamental issue in the study of human intelligence. The question is, in other words, whether high intelligence means a high general ability to learn across distinct domains such as mathematics, language and music or whether it is specialised within such domains.

Strong evidence for a general human intelligence has come from studies analysing performance across multiple domains. Many studies have found that performance correlates across multiple tasks. If, in other words, a person learns mathematics easily then it is likely that the person will also learn languages easily. This shows that there is a single underlying factor, referred to as Spearman’s *g*. Spearman’s *g* does, however, only account for less than 50% of the variance, indicating that there are also parts of human intelligence that are specialised for particular domains. Three domains have been identified by finding correlations across cognitive tests that exist in addition to the correlation accounted for by Spearman’s *g*: numerical, spatial and linguistic.

The current consensus model of human intelligence is a hierarchical model. At the top of the hierarchy sits general intelligence, Spearman’s *g*, which seems to influence our ability to learn across many domains. At the next level sits the three specific abilities, or intelligences, pertaining specifically to numerical, spatial and linguistic learning. Although this model accounts for a relatively large part of the variance in test performance, it does not account for all of it. Therefore, models have been developed containing additional levels in the hierarchy each dividing the domains into several subdomains.

To arrive at a full model of intelligence we must make sure that we have included tests that tap into all relevant subdomains. This is a difficult task and whether we have solved it is still a contentious issue. Take for example the ability to understand other people’s intentions and feelings. This is sometimes referred to as social intelligence. Another example is the learning of motor skills. Some people learn to

swim or juggle four balls more easily than others. Should these abilities be included in our model of intelligence, or should they be considered distinct abilities, or potentials? This depends, in large part, of how we define intelligence. Would we, for example, based on how easily a person learned to swim make judgment about the person's intelligence? Perhaps this would stretch the concept of intelligence too far for many of us. However, we would be more likely to accept a midfielder's ability to read the game and pass the ball in ways that tricks the opponent as a form of intelligence, even though this ability may not be related to numerical, spatial or linguistic skills. These examples show that the domain of intelligence is matter of definition.

Moving on to artificial intelligence, we will soon realise that many of the skills that have proven to be challenging for AI systems seem to fall quite far outside of what would commonly be considered intelligent for humans. Take, for example, walking. Bipedal walking has been challenging to implement in AI controlled robots, because it requires a fast and complex feedback loop that registers small changes in the ground surface and the centre of gravity in order not to fall. Even though we have begun to see robots mastering bipedal walking, the next level challenges of running, jumping or dribbling a ball are still not mastered by AI systems. Yet, we would not consider mastery of these skills by a human as a sign of high intelligence. Other examples include recognising objects in images, comprehending speech and driving a car. These have all been challenges for artificial intelligence but not for human intelligence. On the other hand, human intelligence tests contain parts that could easily be solved by a computer with simple algorithms that are not considered AI. The Wechsler Adult Intelligence Scale, for instance, contain a working memory test, in which the test participant must store a sequence of digits and report them back in reverse order, and a timed arithmetic test, in which the test participant has to solve an arithmetic problem within a time limit. It seems reasonable and valid to include such tests in a test of human intelligence because we would generally consider the ability to solve them as a sign of high intelligence in humans. It is, however, trivial to program a computer to solve them at a much higher level than could ever be achieved by a human even without the use of what we understand as artificial intelligence.

In summary, intelligence, be it human or artificial is a measure of the system's ability to learn across a range of domains. However, the range of domains that we consider related to intelligence, depends on the definition of intelligence and is different for human and artificial intelligence.

1.2 Levels of computation

The human brain computes. Computers as we know them, obviously, also computes. Therefore, it analysing the human brain in a computational framework is an important and influential approach in cognitive neuroscience.

It may seem obvious that the brain computes. After all, the term 'computer' actually originally referred to a profession, or occupation, of doing computations, that were typically repetitive and tedious, for scientific laboratories, observatories or financial institutions. When modern computers were developed, they were called 'electronic computers' to distinguish the machines from humans working as computers. Soon after, the term 'computer' began to unambiguously refer to machines as the occupational title for humans disappeared. As we shall see in this and the following sections, computation is ubiquitous in the human brain and takes many forms beyond tedious, repetitive, formal computations.

You may think that the computational approach to understanding the human brain is somewhat impoverished or simplified. Is the human brain really nothing but a computer? Can complex human abilities like emotions, empathy and intuition really be reduced to computation? Are humans not, unlike computers, conscious? Interesting as these questions may be, we will not dwell on them here. Whether there is more to human cognition and the human brain is left as an open question. The only claim that we will make is that computation is an essential part of what the brain does. We do not aim to show that the brain does nothing but computing.

Brains and computers are, obviously, quite different even though they may both compute. Humans are alive while computers are not. Humans are therefore emergent systems while computers are programmed and designed. Computers and brains have very different structure. Such comparisons can

lead to some confusion unless we define what we mean by *computation* more clearly. David Marr was an early advocate of the computational approach to understanding human cognition. He defined three levels of computation that are helpful for comparing computation in humans and machines.

1. Computational level

The Computational level defines the computational problem that the system is solving including its inputs, outputs, and objective. It is at this level that computation in humans and machines directly are alike. Note that the requirements do not specify *how* the system should be built. This is specified at the algorithmic and implementational levels.

From a computer science perspective, the computational level corresponds approximately to a set of *system requirements*. From a biological perspective, the computational level corresponds closely to a description of cognitive functions or abilities. Marr used the example of constructing a 3-dimensional model of the world given two 2-dimensional retinal images. The model can be used to estimate distances to objects. From a computer science perspective, we can define the computational problem by its inputs (stereo images), its outputs, (an estimate of distance, or depth), and an objective, (minimising the error of the estimate). From a biological perspective, we can observe that a biological system is able to perform the task although this often requires careful control and measurement of the input, output and error.

2. Algorithmic level

The algorithmic level describes *how* the computational problem is solved by describing the underlying algorithm.

From a computer science perspective, the algorithmic level corresponds to computer algorithms and software implementation. From a biological perspective, the algorithmic level corresponds to the algorithms that the brain uses to solve the problem. Staying with Marr's example of constructing a 3-dimensional model of the world given two 2-dimensional retinal images we could, for example, specify that the system should estimate the displacement of the retinal images and base its estimate of the distance to an object using triangulation. From a computer science perspective, we typically have a very clear description of machine learning systems at the algorithmic level because the algorithms are designed and documented by humans. From a biological perspective, it is a challenge to describe human cognition at this level. The problem is analogous to the problem of describing the underlying algorithm of a machine learning system *without* access to the code and with no knowledge of its design. In this sense it is a *reverse engineering problem*. This is the topic of computational cognitive science where computer algorithms are used as models of human computation. We shall present some examples of this approach in later chapters.

3. Implementational level

The implementation level describes the physical realization of the system.

From a computer science perspective, the algorithmic level corresponds to computer hardware. As for the algorithmic level, we typically have a very clear understanding of this level. We could, for example, specify that the triangulation algorithm described above runs on a computer with certain specifications (no dedicated graphics card, that it has 4 GB RAM, etc). From a biological perspective, the understanding of the implementational level comes through computational neuroscience, the study of the brain and how it computes. We could, for example, specify the brain areas involved in the computations. We could also describe how neurons are connected within those areas. Or, we could describe the relative timing of the activation of those areas. All of this will, of course, require measurements of neural activity. We can use this information to build computational models that simulate neural activity performing the computations.

We can now proceed to study the similarities and differences between the key elements of human and machine intelligence: learning and memory.

1.3 Learning

Machine learning, arguably the most influential form of AI, can be divided into three learning paradigms. We can place many aspects of human learning under the same paradigms. This will be a first step in choosing a specific machine learning model that can be compared to human cognition.

- **Supervised learning**

The goal in supervised learning is to learn a function that maps inputs (features) to outputs (labels). The function is learned from a training set of input-output pairs. The objective is to minimise the error between the predicted label and the true label.

Humans supervised learning occur in many domains. One example is image classification. Say we have a set of images, or image features, with each image labelled as portraying a cat or a dog. The goal is to learn to label new, unlabelled images correctly. Another example could be learning the pronunciation of words. Here the input, a word, is symbolic and the output label is a phonetic representation. When studying these types of learning in humans, we can choose supervised machine learning algorithms as models and begin to ask further questions. We can, for example, ask if human classification is better modelled by a linear or non-linear classifier.

- **Unsupervised learning**

The goal in unsupervised learning models is to map inputs to structures such as lower-dimensional representations, clusters or probability distributions. The structure is learned from a training set of unlabelled data. The objective is generally a measure of goodness of the structure and depends on the particular type of unsupervised learning. The goodness can reflect how accurately the input can be compressed and reconstructed, as for Principal Component Analysis (PCA) or auto-encoders. Alternatively, it could be a measure of how tightly similar inputs cluster together and how distinct the clusters are from one another as in k -means clustering. Or, it could be the likelihood of observed data if the structure is a probabilistic model like a Gaussian Mixture Model (GMM) or a variational auto-encoder (VAE).

Humans learn to extract meaningful features from sensory inputs independent of any type of feedback. One example is children learning to segment speech sounds and words from a continuous sound stream. Naively, we tend to perceive pauses of silence in between words in natural speech. Perhaps you have noticed that these pauses are difficult to hear in a language that you do not understand. The reason is that there are no pauses. Children depend on the statistical regularities of language for learning to segment the speech streams. Learning this, depends strongly on exposure to language, but not so much on feedback. Feedback does, of course, play an important role for acquiring language, but only in later stages, when we learn the meaning of words in the language. Another example is our ability to extract structure, such as edges and objects, from visual input. Ample evidence show that this ability depends on receiving varied visual inputs during development. This indicates that it is an ability that is learned, but it does not depend on feedback. In Chapter 2, we will examine how the brain's visual system may employ unsupervised algorithms, like PCA, to learn how to extract meaningful features from visual input.

- **Reinforcement learning**

The goal in reinforcement learning models is to learn a function—called a *policy*—that maps states of the environment to actions. In this sense, reinforcement learning resembles supervised learning, since states can be viewed as inputs and actions as outputs. The key difference lies in the objective: supervised learning adjusts the model to match correct labels for each input, whereas reinforcement learning adjusts the policy to maximize cumulative reward across future states, even when the reward is delayed. The objective is thus defined by expected long-term reward, sometimes modified with a discount factor to weigh immediate and future rewards.

Games such as computer games, chess, and Go are popular test-beds for reinforcement learning algorithms and illustrate reinforcement learning in humans. Unlike many real-world problems,

they have well-defined discrete state and action spaces, which helps defining computer algorithms to solve them. Humans and computers alike are able to learn how to play these games not only by instruction but by trial-and-error. In this case, it should be clear how reinforcement learning differs from supervised learning as the reward, or penalty, is not immediately relatable to a particular action. A chess move that leads to an immediate gain may lead to a future loss.

We can use the division of learning into the three learning paradigms to draw parallels between human and machine learning and to develop models of human learning inspired by machine learning algorithms. Still, some cases of human learning are difficult to place in these paradigms and do not have well-developed machine learning analogies. Imitation and observational learning, for example, is still largely outside of machine learning. An example of this is when children learn language or motor skills simply by observing others with no explicit instructions or reward. Exploratory and intrinsically motivated learning is also difficult to place in a machine learning paradigm. Children's play is a good example of this type of learning. Another example could be artists developing their skills to express themselves despite lack of reward or an externally defined goal. Still, settling for less than a complete computational account of human learning, the three learning paradigms is a good starting point for comparing human and machine learning.

So far, we have described the learning paradigms only at the computational and algorithmic levels. We have waited with a description of the implementational level for a reason: The hardware used in machine learning varies little across the learning domains. Our choice for the configuration of our computers rarely depend on whether we want to develop a supervised or an unsupervised model. The brain, however, contains many distinct modules for learning. A full description is beyond the scope of these lecture notes, but in the next sections we will begin to compare the most fundamental component of learning, memory, at the implementational level.

1.4 Human vs Computer Memory

For humans and machines, there can be no learning without memory. In this section, we will compare human and computer memory. Again, we will see that the comparison is more meaningful, if we compare at each of Marr's three levels of computation.

At the computational level, we can define memory as the ability to encode information, store it for a period of time, and then retrieve it for recall, to use in processing new information or guiding actions.

At the algorithmic level, notice that information takes two distinct forms in the learning paradigms described above. One form is the input data to the algorithms. This form of information corresponds to rote memorisation. For example, you have memorised the letters of the alphabet and the names of your friends. We could store the same information in computer memory. The other form is the model parameters that have been adapted through training the model. This corresponds to skills that have been learned. For example, you can classify an animal in an image that you have never seen before and we could train a supervised algorithm to do the same.

At the implementational level, we find the greatest difference between human and computer memory. Computers store memory in cells that can be in one of two states, effectively representing zeros and ones. At the implementational level, all information is therefore stored in binary form. Images, sounds, or text must first be encoded into this binary form before they can be stored, and later decoded back into usable formats. These encoding and decoding processes typically occur at the algorithmic level. Similarly, machine learning models, including their parameters and weights, are stored in binary, while the actual learning—the updating of parameters—takes place at the algorithmic level. Brains store information differently. Neurons are inter-connected by synaptic connections. It is this structure that inspired artificial neural networks in machine learning. The strength of the connections are similar to the weights in artificial neural networks. In brains, learning thus takes place at the implementational level without a algorithmic layer encoding weights as binary data. Data, such as images, sounds, or text are also encoded into this format and the brain is therefore not well suited to rote memorisation of this type of data. When you remember an image, for example, your brain encodes it as features,

objects and relationships. Your brain is very poor at remembering the image pixel by pixel even for images with very low resolution.

We can arrive at a quantitative estimate of the difference between the capacity of human and computer memory. The largest computer data storage systems are being measured in petabytes, that is 10^{15} bytes. In comparison, the world record for memorising digits of π is around 70.000, which corresponds to approximately 34 kilobytes of data. Measured in this way, the memory capacity of computers thus exceeds that of humans by about twelve orders of magnitude. However, somewhat mysteriously, humans still surpass computers in many aspects of learning.

There are, however, also a striking similarity between human and computer memory at the implementational level: they both divide memory into volatile and non-volatile memory.

Computers can store information in *Random Access Memory* (RAM) and in hard or solid states drives. The reason for this division is that each type of memory has its advantages and disadvantages. The main advantage of RAM is speed, which makes it well suited for storing information that is actively being processed. Hence, we could also refer to it as the *working memory* of computers. Its disadvantage is that it requires a constant supply of energy. This is why it is volatile: all information is lost when the power is turned off. The energy requirement affects, to some extent, how the capacity of RAM is prioritised when building computers. Typically, the capacity of RAM is much smaller than the capacity of hard or solid state drives. Another important reason is that RAM is more expensive to build.

It probably come as no surprise that volatile and non-volatile memory systems are distinct in the brain. Volatile, short-term or working memory is used for information that is actively processed. It requires neural activity and therefore energy, which is a great concern for a biological system. This is one likely reason for why its capacity is limited. The information you can actively process is very limited compared to all the things you remember. Long term memory, in contrast, stores information in synaptic connections as described above. Although this distinction between short-term working memory and long-term memory may seem familiar, or even obvious, the evidence for these memory systems being distinct in the brain has taken decades of research, and our understanding of it is still far from complete. This will be the topic of the next section.

1.5 Human working memory and long-term memory

Several lines of evidence show that working memory (volatile) and long term memory (non-volatile) are distinct components, or *modules*, of human memory. Merging several lines of evidence is an important practise in the study of the brain and cognition and ensures that alternative hypotheses are ruled out. For example, you might think that it is obvious that volatile and non-volatile memory systems exist in the human brain because, obviously, some memories are fleeting while other memories last. However, this does not necessarily mean that the two systems are separate. It could instead reflect a single memory system in which information is stored more persistently than other information.

Some early evidence showing that human memory consist of distinct working memory and long-term memory modules comes from the study of a patient known as HM. HM underwent surgical removal of a brain structure, the hippocampus, to treat his severe epilepsy. After the surgery, HM appeared normal, but it soon transpired that the surgery had left him without the ability to acquire new long-term memories. His working memory was unimpaired, so that he was able to hold a conversation or even play a game of chess. That is why he, at first glance, appeared normal, but he would hold no memory of the conversation or the game of chess after a short while if he was distracted. His long term memory was also intact, so that he was able to recall events that happened before the surgery. This indicates that the mechanism for storing memories long-term can be impaired even if working memory is intact, but could it still be part of a single system? Studies of another patient, KF, who suffered brain injury from a bicycle accident showed that working memory can be impaired without affecting the mechanism that stores memories long term. This shows a *double dissociation* between working memory and the mechanism that stores information long-term. Since we have observed impaired working memory with intact transfer to long-term memory (patient KF) *and* the reverse pattern of impaired transfer

to long-term memory with intact working memory, we now have strong evidence for the two functions being distinct and separable.

Evidence from neuroscience supports the finding that working memory and storing information into long-term memory are distinct functions. When humans hold information in working memory, neurons in certain parts of the cerebral cortex are persistently active. Notably, this sustained activity predicts whether information was retained. In contrast, when humans learn and store information in long-term memory, activation occurs in a different part of the brain, the hippocampus, which is the structure that was excised in HM's surgery.

Additional support has come from behavioural studies, the topic of *experimental psychology*. In the free recall task, participants are presented with a sequence of more than 10 items that they are asked to try to store in memory. This is too many items for most people to keep in working memory, so task performance relies on the participants' ability to store the items in long-term memory. When the sequence ends, participants are asked to recall as many items as they can. Participants can report the items that they remember in any order. Therefore the task is known as the free recall task. When the proportion of letters that were recalled, across several repetitions of the experiment, is plotted as a function of the item's position in the presentation sequence, two interesting effects can be seen. Items in the beginning of the sequence are recalled better than items in the middle of the sequence. This is called the *primacy effect*. The primacy effect reflects that the task is easier in the beginning of the sequence because the number of items that participants are storing is still low. Hence, participants can allocate more time to storing each new item in long-term memory. As the number of items increases through the sequence, the task becomes more difficult and performance suffers. Also, items at the end of the sequence are recalled better than items in the middle of the sequence. This is called the *recency effect*. The recency effect reflects that items at the end of the sequence do not need to be stored in long-term memory since they can be held in the short-term working memory. Control experiments have verified this interpretation of the primacy and recency effects. First, increasing the rate at which the items are presented decreases the primacy effect because participants will now have less time to dedicate to storing the first items in the sequence. This, however, does not affect the recency effect because the items can still be stored in working memory as well as when the rate of presentation was slower. Secondly, adding a working memory task to the experiment, right after the sequence ends, before the participants begin to recall it, decreases the recency effect because participants cannot hold the last items in working memory when it is occupied by another task. Notably, a delay with no working memory task, does not affect the recency effect because it does not interfere with the participants' working memory. This shows that the effect is due to the working memory task and not simply due to the delay. As a final observation, the working memory task, does not affect the primacy effect because the first items in the sequence have already been stored in long-term memory.

In summary, converging lines of evidence show that human working memory and long-term memory can be considered distinct, functionally segregated modules. We have, however, only scratched the surface in describing the complex architecture of human memory.

1.6 Human working memory

A surprising fact stemming back from the earliest studies of human working memory is that its capacity is quite limited. In serial recall tasks, a sequence of letters or digits are presented to observers who then tries to recall the sequence in the correct order. Surprisingly, most observers struggle with recalling sequences of more than seven items. We can translate that into bytes if we assume that the items are letters in the English alphabet. As there are 26 letters in the alphabet, $2^5 = 32$ states will suffice to define a letter, which means that each letter requires 5 bits. A memory with a capacity to hold seven letters therefore corresponds roughly to 35 bits, or a little less than 5 bytes. Measured in this way, which we will come to see as an over-simplification, the capacity of human working memory is approximately 13 bytes. This is about nine orders of magnitude smaller than the capacity of the volatile memory in standard computer, which is typically measured in gigabytes. Compare this to our mental experience of the complex world around us. When we walk into a room full of people and objects, we

experience that we immediately perceive it all quite clearly. Even when we exit the room, we often have a sense of being able to recollect the experience quite clearly. This, however, is quite far from the truth. Many experiments show that we fail to notice quite drastic changes in a scene after a brief intermission—a phenomenon known as change blindness. In one striking demonstration, a conversation was disrupted by a couple of men carrying a door and walking in between the interlocutors. Unknown to one of the interlocutors, the unknowing participant in the experiment, the other was swiftly exchanged with another person, which often went unnoticed by the unknowing participant in the experiment, demonstrating our very limited working memory capacity. This leaves us with the question of how human behaviour can be so complex compared to that of computers when our working memory capacity is so limited in comparison.

One clue for why human working memory is not quite as limited as estimated above comes from a control experiment in which, instead of single letters, each item is a three letter word like *cat*, *dog*, *job*, or *eye*. In this experiment, we will find that the capacity is *still* approximately 7 items. This is surprising because, measured in bytes, we have clearly tripled the information load. Had we instead used random consonant triplets rather than three-letter words in the memory test, humans would have been able to hold far fewer items in working memory. This reflects the inherently associative nature of human memory: it stores patterns—often called *chunks* in the cognitive science literature—rather than isolated bits. These patterns can more complex than three letter words. The upper limit is elusive and depends on the observer’s prior experience and context, since items in working memory function as *pointers* to patterns stored in long-term memory.

Working memory can be shown to consist of sub-modules. When letters are used in serial recall tasks participants tend to make certain types of errors. Specifically, they tend to confuse letters that sound alike (such as ‘F’ and ‘S’) rather than letters that look alike (such as ‘F’ and ‘E’). This happens even though the letters are presented visually so it is unlikely to be because the letters are misperceived. Instead, it indicates that the letters are encoded as sounds in working memory. Why would observers recode letters presented visually to sounds? Although introspection is not a scientifically valid method because the observations cannot be shared, we can still use it as inspiration. Trying a serial recall task of remembering a random sequence of letters for a short period of time may shed some light on the mechanics of working memory: we tend to repeat the sequence to ourselves in the retention period. This is sometimes called articulatory rehearsal. We often do this quietly with a silent inner voice, without moving our mouth and tongue, a phenomenon called subvocalisation. Articulatory rehearsal can be considered a sub-module of working memory, sometimes referred to as a *slave system*, because of its passive and repetitive character, hardly deserving the term working memory. The use of articulatory rehearsal in this way has been confirmed by behavioural experiments. In one experiment, participants were prevented from using articulatory rehearsal by requiring them to vocalize an irrelevant sequence of sounds (for example, “tah-dah-tah-dah...”). This procedure, known as articulatory suppression, disrupts the phonological loop and eliminates the usual advantage of articulatory rehearsal. As a result, memory span for verbal material is greatly reduced, from approximately seven to four items, demonstrating that articulatory rehearsal is a key mechanism for maintaining verbal information in working memory. Furthermore, an analysis of the type of errors that the participants made showed that they did not tend to confuse letters that sound alike rather than letters that look alike during articulatory suppression, which lends further support for the effects of articulatory rehearsal. Finally, to show that the decrease in working memory capacity is not simply an effect of a secondary task, and not specific to an articulatory task, another experiment showed that other tasks, like finger-tapping, does not cause a decrease in working memory capacity.

Since the early description of working memory and articulatory rehearsal, two additional slave systems, the visuo-spatial sketchpad and the episodic buffer have been shown to also aid working memory. As their names imply, these systems are specialized: the visuospatial sketchpad stores visual and spatial information, while the episodic buffer maintains an integrated “story-line” of the recent past by binding information across modalities and linking it to long-term memory.

We have now described how human working memory consists of multiple specialized components and stores patterns of varying complexity rather than raw information. In this sense it differs fundamentally from volatile computer memory. These differences arise at the implementational level. For brains,

however, the distinction between the implementational and algorithmic levels is vague because the hardware is so specialised for the algorithm. For computers, software systems such as machine learning models are needed before they can mimic some of the ways in which human memory stores and processes information.

1.7 AI mimicking human cognition

AI systems are often built to mimic human cognitive functions. Examples include speech recognition and some image classification tasks. It can be tempting to consider this type of problems as reverse engineering problems where a functioning system—humans in this case—are available, so that we can try to construct the engineering solution—an AI system in this case—to work in the same manner. History carries a few lessons that show that this is not always the best approach. One lesson comes not from the history of AI but from the history of human flight. The earliest pioneers of aviation attacked the problem as a reverse engineering solution attempting to build flying machines mimicking the flapping wings of birds. We now know that aircrafts, with fixed wings and the power of combustion engines, unparalleled by nature, is the better solution. Another example that comes from the history of AI, is speech recognition. Where some early solutions tried to mimic the function of the human auditory system, more recent and successful applications are based on neural networks that are only vaguely inspired by the human brain and owe, in large part, their success to being trained on a much larger speech corpus than that needed by a human child to acquire speech. Hence, although the reverse engineering approach may inspire the development of AI systems it may also be an unnecessary constraint.

Mimicking human cognition require AI systems to be trained on behavioural data, i.e. data that somehow records human behaviour. One example is AI systems used to classify electroencephalography (EEG) data. EEG is typically recorded by measuring the changes in the electrical field from electrodes placed on the human scalp. These changes are influenced by neural activity but also by eye movements and muscle activity. In order to filter out the activity from non-neural sources human experts visually inspect the data and classify it as stemming from neural or non-neural activity. This is a tedious, slow and expensive process and machine learning approaches has therefore been developed to solve the problem. The data available to train the algorithms typically consists of samples of EEG data, which has been labelled by human experts. Therefore, the data does not actually contain the ground truth, i.e. a label indicating whether the sample actually contained data reflecting neural activity or not. Instead, the algorithm is trained to mimic human experts including the mistakes that they make. Therefore, we cannot hope for the algorithm to perform better than human experts, even though this might be possible if ground truth data were available. The performance of human experts is often quantified as the inter-rater agreement, i.e. the rate at which the human experts agree with one another. We can use this measure as benchmark that we can use to assess the performance of AI systems.

Another example of AI systems mimicking human behaviour is visual object recognition. Here, the samples consist of images and labels produced by humans labelling images as depicting, say, a traffic light or a dog. Large sample sizes are often needed to train an AI system and the labelling is therefore costly even though it a single image can easily be labelled by a human. It therefore takes care and consideration to select these samples. It is, for example, important to check that the humans are really trying their best when performing the task. Also, the samples must be representative of samples that the AI system will meet when deployed. Many examples show that object recognition systems can be fooled by images that were unmatched in the training samples. One example showed that a small sticker on a road sign would cause some AI systems to mis-classify the road sign as another type of object. Humans, on the other hand, would not be so easily duped. Carefully selecting the samples to represent the scenario in which the AI system will be deployed would allow it to better mimic the powerful abilities of human cognition.

The inability of many AI systems to generalise to data that are outside the training data distribution, challenges the concept that AI systems are intelligent. Imagine a human that had been trained to recognise traffic signs but had never seen a traffic sign with a small sticker patched onto it. Humans

would be very unlikely to make the mistake of thinking that the traffic sign is not a traffic sign. The reason for this is probably that human visual perception is interlinked with a general understanding of the world around us, something that AI systems generally lack. In this respect, humans are still superior to AI systems. The superior human ability to generalise is, however, not just due to our vast general knowledge of the world. Children, learn to classify, say, animal species with far fewer samples than the number required by AI systems. This difference deserves further study if we aim to develop better and more general AI rather than just developing AI solutions that work for specific problems.

1.8 Mini project

In this mini project will work in groups of 2-4 students. You will conduct the behavioural experiments in an attempt to replicate the findings experiments described in Section 1.5 and 1.6 including the control experiments.

Specifically, you should run the following experiments and try to replicate the effect listed under each experiment.

Free recall experiment effects

- The primacy and recency effects
- The effect of increasing the rate at which the items are presented
- The effect of adding a working memory task to the experiment, right after the sequence ends, before the participants begin to recall it
- The effect of adding a pause to the experiment, right after the sequence ends, before the participants begin to recall it

Serial recall experiment

- The limited capacity of working memory
- The type of errors that participants make
- The effect of chunking
- The effect of articulatory suppression
- The (lack of) effect of finger tapping

First, design the experiments. Some of the things that you need to decide on are

- Whether you want to use digits, letters, words, or something else as the items that participants memorise
- The rate at which the items are presented
- How participants respond
- How you calculate the proportion of correct responses from the responses
- How you compare the proportion correct within and across experiments. You should include some error measure, like confidence intervals, in your comparison. Recall, that if the confidence intervals are overlapping then there is no difference
- How many repetitions (at least 20) of the experiment each participant performs

When you have designed the experiments, you should write a script that can run the experiment. Each member of your group should run the experiment. Once you have the data, then you should analyse the results. You can pool the data across the participants meaning that if you have three participants running each experiment 20 times then you can treat the data as if a single participant had run the experiment 60 times. This is a simplification and not the proper way of analysing the results, because it ignores the variability across participants, but the full analysis is beyond the scope of this course. Based on your analysis, discuss whether you were able to replicate the findings described in the main text.

Your report should be no more than 5 pages including figures. Your report will be evaluated on the following elements

- The description of your experimental design
- The description of your analysis of the results
- The discussion of whether your results replicate the experimental results described in Section 1.5 and 1.6 including the control experiments
- The discussion of why your results might not replicate the results described in Section 1.5 and 1.6 including the control experiments

Note that you will *not* be evaluated on whether you actually manage to replicate the results.

The free and serial recall tasks are very well described in the scientific literature. You may use scientific articles, popular articles on the internet and large language models, like ChatGPT and Microsoft Copilot to help you designing the experiments. You may also use large language models to assist you in coding. You may *not* use large language models to write your report except for proof reading and help with grammar and vocabulary.

CHAPTER 2

Neural encoding

2.1 Linear encoding

Neurons are the main information processors in the brain. They come different forms and the details of their functioning are, unsurprisingly, complex. We can, however, come to a basic understanding of how neurons encode information from a very simplified linear model.

Neurons generally have dendrites that derive their name from their tree-like appearance. The dendrites are dotted with receptors that can be activated by specific neurotransmitters sent from the axons, or nerve fibres, of other neurons. The dendrites thus receive input from other neurons while the axons send an output to other neurons.

The dendrites of a single neuron can receive input from many (thousands) of other neurons. All these input do not affect the neuron in the same way. Some affect the neuron more strongly than others. The effect can furthermore be either excitatory, which means that the input increase the activation level of the receiving neuron while others are inhibitory and have the opposite effect. The input to a neuron is thus *weighted* meaning that it is multiplied by a real scalar. If a neuron receives input from N neurons, a simple model for the activation level, a_l , is the linear expression

$$a_l = \mathbf{j}^T \mathbf{w}_l \quad (2.1)$$

where $\mathbf{j}$ is an N -dimensional vector containing the input the neuron receives and $\mathbf{w}_l$ is a N -dimensional vector containing the weights of the input to the neuron.

Extending to L neurons each receiving the same input, $\mathbf{j}$, the linear model can be written as

$$\mathbf{a}^T = \begin{bmatrix} a_1 \\ \vdots \\ a_L \end{bmatrix}^T = \mathbf{j}^T [\mathbf{w}_1 \quad \cdots \quad \mathbf{w}_L] = \mathbf{j}^T \mathbf{W} \quad (2.2)$$

Here, we will work with the simple linear model of neural encoding given by Equation 2.1. This model is an over-simplification in comparison to both biological neurons and the artificial neurons in neural networks that are used in machine learning. Note, that unlike a networks of these more complex, non-linear neurons, a network of linear neurons does not gain complexity from having multiple layers. To see why this is so, consider a two-layer linear network where the output of the first layer is $\mathbf{j}\mathbf{W}_1$. This output serves as the input to the second layer through weight matrix $\mathbf{W}_2$. We can then write the activation of the second layer as

$$\mathbf{a} = \mathbf{j}\mathbf{W}_1\mathbf{W}_2 = \mathbf{j}\mathbf{W}_{12}$$

Hence, a single layer model with weight matrix $\mathbf{W}_{12} = \mathbf{W}_1\mathbf{W}_2$ is mathematically equivalent to the two-layer model.

Biological neurons become active and send an action potential down their axon only when they receive activation above a certain threshold. The action potential causes the release of neurotransmitters that activate other neurons. The action potential is binary, often referred to as all-or-nothing in the biological literature. However, this does not mean that neurons are binary like a simple on-off switch or a bit because the *rate* at which action potentials occur is very important for how they activate other neurons. If a neuron is activated far above its threshold, then it will fire at a high rate, activating

other neurons more strongly than if it were activated only slightly above its threshold. We can model this using an activation function, but first we should clarify a somewhat confusing terminology: In the neurobiological literature, the activation level of the neuron describes how much it is activated. This is the activation described by Equation 2.1. This activation level is the input to the activation function. The output of the activation function is the degree to which the neuron activates other neurons. Here, we will not provide an exact activation function for biological neurons, but only note that the output of the function is zero (or has a constant baseline value) when the activation level is below the threshold, it increases with increasing activation above the threshold, and it is limited by a maximum since biological neurons will naturally have a maximum firing rate.

Neurons in artificial neural networks also employ an activation function. In the neural network literature, activation typically refers to the *output* of the activation function, which is, confusingly, not the same usage of the term as is found in the neurobiological literature. Commonly used activation functions include sigmoid (s-shaped) activation functions such as the hyperbolic tangent function and the logistic function. These functions are monotonically increasing and have an upper bound, but they are different from the activation function of biological neurons in that they do not have well-defined threshold under which the function takes on the value of zero (or a constant baseline level). The rectified linear unit (ReLU) activation function is also commonly used. It does have a well-defined threshold and, above this threshold, it is monotonically increasing, but it is different from the activation function of biological neurons in that it has no upper bound.

Biological neurons and neurons in artificial neural networks both rely on non-linear activation functions. This non-linearity makes them more flexible, allowing them to model both linear and non-linear input-output relationships. This flexibility is key to their effectiveness in information processing. However, this same flexibility has also contributed to the challenge of understanding how artificial neural networks process information and how to use them effectively. While non-linear neural networks gained significant attention in the 1980s, they only gained wider success the 2010s.

Despite the importance of non-linear activation functions for information processing in both biological and artificial neural networks, we focus exclusively on linear encoding here. One reason for this is that it is often difficult to qualitatively understand how non-linear neurons process information. In the machine learning literature this is known as the problem of *explainability*. Another reason is that although non-linear information processing is an important aspect of how the brain works, it is still possible to develop an understanding of information processing in the brain using linear models, as we shall see in the following sections.

2.2 Lateral inhibition

The first stage of encoding visual information happens in the photoreceptors in the retina of the eye. These cells are specialised *sensory* neurons that do not have dendrites that allow them to be activated by other neurons. Instead they have similar structures that allow the cells to be activated by light. There are two types of photoreceptors: rods and cones. Rods are more sensitive to light than cones. Therefore they are more important for our vision at low light. Cones are less sensitive to light. There are three types of cones. Each of these three types are more sensitive to light at a particular part of the visual spectrum. The combination of information from each type of cone is the basis of colour vision. In this section, we shall not consider colour vision, for simplicity. The photoreceptors are thus comparable to pixels in gray-scale digital images and their activation levels represent the brightness level of the a small area of the retina.

The second stage of encoding occurs also in the retina before information transmitted to the brain. In the retina, the axons of the photoreceptors project onto bipolar cells. A bipolar cell will be activated by a photoreceptor but, at the same time, it will be inhibited by the neighbouring photoreceptors. This simple network structure is called *lateral inhibition* and is repeated across the retina.

Lateral inhibition gives the bipolar cells an interesting property: they encode visual contrast rather than brightness. To see why this is so imagine that light shines on all the photoreceptors that project onto a bipolar cell. The activation level of the bipolar cell will increase due the excitation it receives from

one photoreceptor but, at the same time, it will be inhibited by the other neighbouring photoreceptors. If we set the weight of the excitatory cell to 1 and the weight of the inhibitory cells to w then we can write the one-dimensional case as

$$a_m = wj(m-1) + j(m) + wj(m+1) = \mathbf{w}^T \mathbf{j} \quad (2.3)$$

where $\mathbf{w}^T = [w \ 1 \ w]$ and $\mathbf{j}^T = [j(m-1) \ j(m) \ j(m+1)]$.

Note that, the last term in Equation 2.3 is the same as in Equation 2.1. Lateral inhibition is thus a form of linear encoding. Note also that the input vector, $\mathbf{j}$ must be two elements longer than the activation vector, here a scalar a_m . This is because we do not have a model for how to calculate the activation of the cells that receive input from the edge of the image. Therefore, we will ignore those cells in the following. Also note that if we set the weight of the inhibitory cells to $w = -0.5$ then the activation will be zero, $a_m = 0$ if the input is constant, $j(m-1) = j(m) = j(m+1)$. Hence, the neuron will not respond to an isoluminant input, but only respond to changing input, i.e. contrast. The bipolar cells thus encode local contrast and the information processing this simple network can thus be thought of as contrast enhancement.

We can see the function of the lateral inhibition network in another way if we rephrase Equation 2.3 as

$$a_m = \sum_{n=-1}^1 j(m+n)w(n) \quad (2.4)$$

Equation 2.4 describes lateral inhibition as a *convolution* of the input with a kernel, $\mathbf{k}_1^T = \mathbf{w}^T = [w \ 1 \ w]$. The network can thus be thought of as a spatial filter where the kernel has been chosen to enhance contrast.

Perceptual illusions are often a striking way to demonstrate the effects of neural information processing and the effects of lateral inhibition is no exception. Consider the Mach band illusion in the top panel of Figure 2.1. At the location of the rightmost arrow you can see a bright vertical band. At the location of the leftmost arrow you might also be able to see a dark band although it is less obvious than the brighter band. You might better see the illusory bands if you change the size of the figure or your viewing distance. The illusion might also be clearer if you do not look directly at the bands. Both bands are illusory. The horizontal luminance profile of the image is shown in the middle panel of the figure: it is a linear transition between the darker area in the left of the image to the brighter area in the right of the image. The vertical luminance profile is constant.

Why do we see the illusory Mach bands? Lateral inhibition may be part of the answer. The activation of a horizontal layer of neurons, that receive lateral inhibition, is plotted in the lower part of Figure 2.1. Notice that the activation is lowest near the location of the darker Mach band and highest near the location of the brighter Mach Band. This is because the image *changes* more at the locations. It is these changes, i.e. contrast, that our visual system tends to exaggerate through lateral inhibition.

Note that when we analyse images such as the Mach band image in the top panel of Figure 2.1 in terms of lateral inhibition, we might not get the expected results if we expect that each pixel activates only one single photoreceptor. If, for instance, each photoreceptor is activated by two pixels in the image then we should choose our kernel to be $\mathbf{k}_2 = [w \ w \ 1 \ 1 \ w \ w]$. If each photoreceptor is activated by three pixels in the image then we should choose our kernel to be $\mathbf{k}_3 = [w \ w \ w \ 1 \ 1 \ 1 \ w \ w \ w]$ and so forth. We will say that these filters has a *scale* of two and three, respectively. For each image, We will have to adjust this scale of the filter in order to get the expected activation levels. The activation profile in the the bottom panel of Figure 2.1 was, for example, created by filtering the image in the top panel with kernels of varying scales and then selecting the scale that gave an activation profile that approximates the perceived illusion.

Images and the retinal array of photoreceptors are, of course, two-dimensional. In two dimensions, we can still use convolution and spatial filtering as a model of lateral inhibition but we must use a matrix kernel. The kernels at scale 1 and 2 are

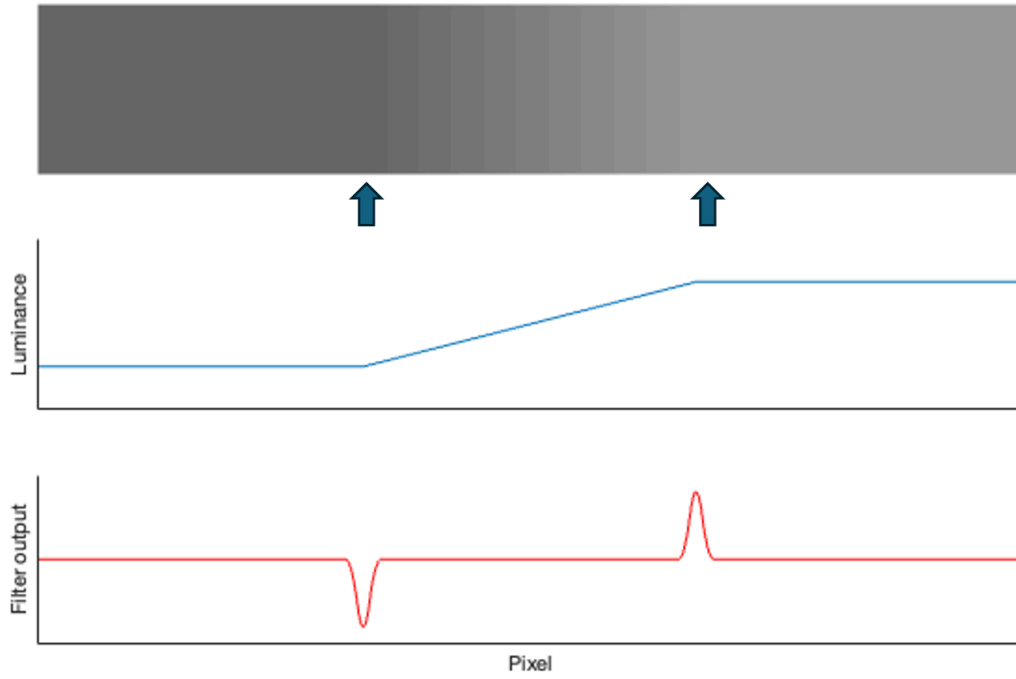


Figure 2.1: In the Mach band illusion in the top panel we perceive a dark and a bright band at the location of the two arrows. The luminance profile plotted in the middle panel does, however not change at the location of the illusory bands. It is a simple linear gradient between a dark and a lighter intensity. The activation profile of a layer of cells that receive lateral inhibition, plotted in the middle panel, is lowest at the location of the dark Mach band and highest at the location of the bright Mach band. This shows that lateral inhibition can explain, in part, the illusion.

$$\mathbf{K}_1 = \begin{bmatrix} w & w & w \\ w & 1 & w \\ w & w & w \end{bmatrix} ; \quad \mathbf{K}_2 = \begin{bmatrix} w & w & w & w & w & w \\ w & w & w & w & w & w \\ w & w & 1 & 1 & w & w \\ w & w & 1 & 1 & w & w \\ w & w & w & w & w & w \\ w & w & w & w & w & w \end{bmatrix} \quad (2.5)$$

Note that if we still want the filter to give an output of zero when the input is constant in the two-dimensional case, the weight the inhibition should be adjusted to $w = -0.125$. Note also that convolution in two dimensions will give the same edge effects as convolution in one dimension: At a scale of 1, the image must be 3-by-3 pixels and at scale of 2, the image must be 6-by-6 pixels. Finally, note that the kernel matrix, $\mathbf{K}$ is *not* equivalent to the weight matrix, $\mathbf{W}$, in Equation 2.2. If we want to write two-dimensional convolution in terms of the linear model, we must first flatten the image and kernel matrices, $\mathbf{J}$ and $\mathbf{W}$. Only then can we use Equation 2.2 to describe convolution in two dimensions. Although the two approaches are mathematically equivalent, it is more practical to use software convolution functions to calculate the activations across the entire image.

The left image in Figure 2.2 shows a photograph of da Vinci's original painting of Mona Lisa. The center image shows the original image after being filtered with a kernel with appropriate scale. Notice how the areas with high contrast have been enhanced. This is seen in particularly at the border between Mona Lisa's dark hair and the brighter sky behind. It is also seen at the border between Mona Lisa's

fair hands and her darker dress. Notice also the edge effects at the edges of the image. These arise because the entire original image was used to calculate the activations in the center image.

As a final stage of processing, we can move beyond the linear model and apply a threshold, so that all activations above a certain threshold are given a value of 1 and while all the other activations assume a value of 0. The result, shown in the right image in Figure 2.2, illustrates that the effect of thresholding is similar to *edge detection*. The image now resembles a line drawing with only the most prominent lines visible.



Figure 2.2: Left: A photograph of da Vinci's original painting of Mona Lisa. Center: When filtered using a kernel of appropriate scale the contrast is enhanced. This is seen in particularly at the border between Mona Lisa's dark hair and the brighter sky behind. It is also seen at the border between Mona Lisa's fair hands and her darker dress. Right: When filtered image is thresholded it resembles a line drawing showing that lateral inhibition followed by thresholding works as *edge detection*.

Simple cells in the visual cortex lie further up in the processing pathway of the visual system. These cells can have more complex response patterns. Some simple cells respond particularly strongly to edges with a specific orientation. This type of behavior can be modelled by a simple network structure where a number of bipolar cells that lines up in the retina excites a simple cell. The example in Figure 2.2 illustrates this.

We will end this section on lateral inhibition by examining another visual illusion, the Hermann grid illustrated in the right panel of Figure 2.3. When looking at the grid you can notice light gray shadows at the intersection of the white lines. Strangely, the shadows are only apparent when you do not look directly at them. This is the Hermann grid illusion. Lateral inhibition can, in part, explain the illusion: At the intersection of the white bars, the bipolar cells receive inhibition from the top, bottom, left and right because these areas are white. In comparison, in a vertical bar away from intersections, the bipolar cells receive inhibition only from the bottom and the bottom. That additional inhibition that the bipolar cells receive at the intersections is what makes them appear less bright than the rest of the bar, hence the gray shadows. The effect can be mimicked by convoluting the original image with a kernel of appropriate size. The convoluted image, will indeed have show the gray shadows at the intersection of the wite lines. This is shown in the left panel of Figure 2.3. So, why do the gray shadows disappear when you look directly at them. The reason may lie in that the density of photoreceptors is much higher in the *fovea* of the retina, which is at the focus of our gaze. In the periphery of the retina, the density of photoreceptors is much less. Hence, the spatial resolution of peripheral vision is much lower, which is why it is very difficult to read text if you do not look directly at it. In terms of lateral inhibition, this means that the kernel size is larger for peripheral vision because more there are fewer photoreceptors per pixel, which means that the kernel size should be larger. Conversely, for foveal

vision, the kernel size should be smaller. In agreement with this, the gray shades do indeed vanish if the kernel size is made smaller than the one used in the right panel of Figure 2.3.

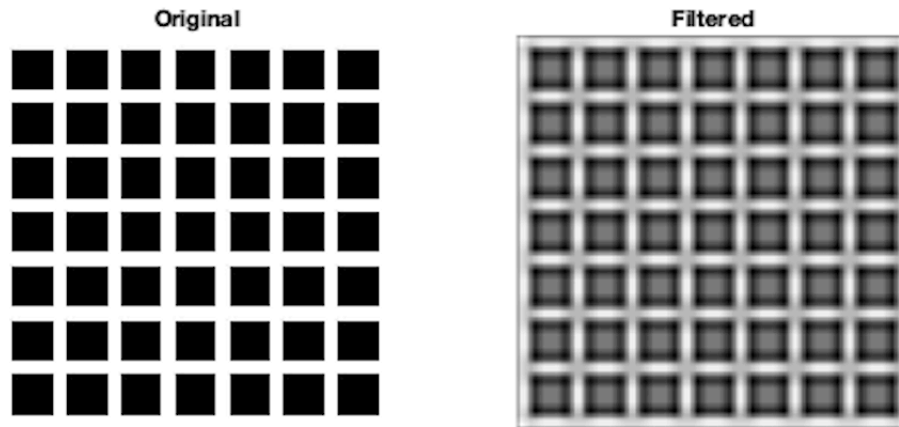


Figure 2.3: Left: The Hermann grid illusion. Gray shades seem to appear at the intersection of the white lines when you do not look directly at them. Strangely, they seem to vanish when you look directly at them. Right: The original image filtered by a kernel with appropriate scale. Notice how the filter has created gray shades at the intersection of the white lines. The activation thus mimicks the Hermann grid illusion.

2.2.1 Exercise - Lateral inhibition

The purpose of this exercise is to show that it is possible to construct simple quantitative models of neural systems and that these models can inform us about brain function and perception. In this exercise, we will examine a simplified model of lateral inhibition in the retina.

1. Assume that we have a 1-dimensional array of M cells. Each cell has an activation level a_m where $m = 2, \dots, M+1$. Each cell receives input j_n from photoreceptor stimulated by light. In addition, each cell receives scaled inhibitory input from neighboring photoreceptors so that

$$a_m = j_m + w(j_{m-1} + j_{m+1}) \quad \text{for } m = 2, \dots, M+1 \quad (2.6)$$

Calculate the activations when input is a 1-dimensional edge, i.e. a steep transition from dark to light. You can use an input vector $\mathbf{j}^T = [0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1]$ and a weight, $w = -0.5$. Plot the values and explain how you can see that the network enhances contrast.

2. We can express Equation 2.6 as

$$a_m = \sum_{n=-1}^{n=1} j(m+n)k(n) \quad (2.7)$$

where k is a discrete kernel function with the elements $k(-1) = k(1) = w$ and $k(0) = 1$. Equation 2.7 is the formula of a 1-dimensional convolution showing that the function of the network is to convolve the input with the kernel, k . Recalculate the activation levels using one-dimensional convolution. There are functions for this in most software for scientific computing. Show that you get the same activation values as you got before.

3. Neurons are only active when the sum of the activation they receive from other neurons exceed a certain threshold. Use a threshold function on the activations so that the network works as an edge-detector in one dimension. Which value do you set the threshold to? Explain your choice.
4. Recreate Figure 2.1. Explain your approach. Describe how you created the luminance profile in the middle panel of Figure 2.1. Which kernel scale did you use?
5. In two dimensions, the neurons would receive inhibitory input from neighboring photoreceptors to the left and right and above and below. This would turn the kernel function into a matrix, $\mathbf{K}$, as described in Equation 2.5. Load the image of Mona Lisa that comes with this exercise and use 2-dimensional convolution to recreate Figure 2.2. Decide which kernel size works best for contrast enhancement and which threshold values works best for edge detection. Explain your approach.
6. Load the image of the Hermann grid that comes with the exercise and use 2-dimensional convolution to recreate Figure 2.3. Decide which kernel size works best for recreating the figure. Explain your approach.

2.3 Principal Component Analysis (PCA) in linear encoding

In Equation 2.2 we described linear encoding of N inputs into L neurons through a matrix, $\mathbf{W}$, of weights. In this section, we will use PCA as a model for how the number of neurons, L , and the values of their weights is determined.

Before we delve into the details of PCA we will consider how to construct the network in more general terms. First, we will notice that it will be good to minimise the number of neurons, L , because neurons take up energy, limited by the food supply, and space, particularly limited within the protective enclosure of the skull. Second, it also seems reasonable that we will need as many neurons as we have inputs, $L = N$, to avoid loss of information. We may, however, accept some loss of information to limit the number of neurons required. In machine learning terms, this is the problem of reducing the dimensionality of the input with as little loss of information as possible.

To minimise the number of neurons, L , they should not encode redundant information. This means that their activation levels across many stimuli should be independent. If the distribution of the activations levels are Gaussian, this means that activation levels of the neurons should have zero correlation. Notably, many studies have shown that this is *not* the case for the inputs. One way to show this is to simply take a digital camera and record many images. The intensity values of neighbouring pixels will be correlated across the images. This simply reflects the structure of our environment that is made up of large surfaces, or homogeneous patches. Most often, the image such a surface or patch will cover multiple photoreceptors although, occasionally, an edge or some form of local contrast will cause neighboring photoreceptors to have very different activation levels. Based on these observations, it seems that we might be able to find a network structure with fewer neurons than inputs, $L < N$, while minimising the loss of information by getting rid of the correlation between the inputs. PCA can do exactly that.

We will not go through all the details of PCA as this is covered elsewhere, e.g. Chapter 3 in Herlau, Schmidt and Mørup Schmidt (Herlau et al., 2021). In stead we will focus on relating PCA to neural encoding.

PCA takes an M -by- N data matrix, in our case an matrix of images, $\mathbf{J}$, where each row is an observation, in our case an image, and each column correspond to a feature, in our case a pixel. The matrix must have zero mean.

PCA factorises this matrix into two matrices, $\mathbf{A}$ and $\mathbf{W}^T$, so that

$$\mathbf{J} = \mathbf{A}\mathbf{W}^T \quad (2.8)$$

In PCA terminology, the columns of $\mathbf{W}$, contains the *principal components*, or *loadings*. The principal components are vectors that form a new basis in the original data space. If the data matrix

contains at least as many observations as features, i.e. if $M \geq N$, then $\mathbf{W}$ is a square N -by- N matrix, so that the number of principal components is equal to the dimensionality of the original data space, which means that the principal components form a complete orthonormal basis. In terms of the neural model of linear encoding, this means that the number of neurons, L , is equal to the number of inputs, N . In that case, $\mathbf{A}$ is an M -by- N matrix in which each row $\mathbf{A}$ contains an image represented in this basis. Notably, the columns of $\mathbf{A}$ are orthogonal. In terms of the neural model of linear encoding, this means that the activation levels of the neurons are not correlated across the images. Since the principal basis is complete, this representation of the data is also complete meaning that we can reconstruct the data matrix $\mathbf{J}$ from the matrices $\mathbf{A}$ and $\mathbf{W}$ without any loss of data. Concordantly, matrices $\mathbf{A}$ and $\mathbf{J}$, are of the same size.

To better understand how PCA relates to linear encoding, we can isolate the matrix $\mathbf{A}$. Since $\mathbf{W}$ is orthonormal, $\mathbf{W}^T \mathbf{W} = \mathbf{I}$, we can isolate $\mathbf{A}$ by right multiplying Equation 2.8 by $\mathbf{W}$

$$\mathbf{A} = \mathbf{J}\mathbf{W} \quad (2.9)$$

If we extract a single image (a row) from the matrices $\mathbf{A}$ and $\mathbf{J}$ then we get the model of linear encoding in Equation 2.2 with one noticeable exception: In Equation 2.2, the weight matrix, $\mathbf{W}$, was N -by- L , where N is the dimensionality of the input (the number of pixels) and L is the number of neurons that receive this input. In Equation 2.9 the weight matrix is N -by- N meaning that the two models are only equivalent if the number of neurons that receive the input is equal to the dimensionality of the input, $L = N$.

PCA is often used for dimension-reduction in machine learning. The orthogonality of the columns of $\mathbf{A}$ makes it particularly suited for this. In statistical terms, the orthogonality means that the data in this representation has zero covariance. Informally, another way to think of this is that all variance that can possibly be contained in one column of $\mathbf{A}$ is indeed contained in that column. The objective of minimising the covariance between the columns is thus equivalent to maximising the variance of each column. Furthermore, PCA sorts the columns of $\mathbf{A}$ by the amount of variance they contain, so that the first column of $\mathbf{A}$ contains the most variance and the last column the least. Therefore, we can reduce the dimension of the neural representation, $\mathbf{A}$, from N to L , by selecting only the first L columns of $\mathbf{A}$ and the corresponding first L principal components (columns of $\mathbf{W}$) knowing that we have kept as much variance as possible in L dimensions.

The mathematical equivalence of the linear neural network in Equation 2.2 and Equation 2.9 allow us to interpret PCA in terms of neural encoding. Each principal components, i.e. each column of $\mathbf{W}$ correspond to a set of neural weights for one neuron. Each row in $\mathbf{A}$ represents the data, i.e. an image, as activations of the neurons.

There is an important difference between Equation 2.8 and Equation 2.9 that their mathematical equivalence may hide. As we have already noticed Equation 2.9 describes neural encoding. Knowing the input, $\mathbf{J}$, and the neural weights, $\mathbf{W}$, we can calculate the activation, $\mathbf{A}$ of the neurons. Equation 2.8 describes neural *decoding*. Knowing the activation of each neuron, $\mathbf{A}$, and the neural weights, $\mathbf{W}$, we can calculate the data, i.e. the image, from the neural representation. This is an important subsequent stage of neural processing. There is no point in only encoding the information; the brain must also be able to decode it so that it can act on it.

In the previous section we saw how a linear network with lateral inhibition performs a convolution, or spatial filtering, of images using kernels like the ones in Equation 2.5. We also briefly discussed how the combination of the output of this filter can be equivalent to detecting edges in images and that neurons in the visual cortex do indeed respond to edges of a particular location and orientation. How does the brain decide to use this particular type of filters? Whereas lateral inhibition in the retina may be *hard wired*, meaning that the development of this network structure and corresponding kernels is genetically pre-determined, we know that the network structure and corresponding kernels in the cortex are determined during development and requires varied visual input. One famous study demonstrated this by raising kittens in an environment with only horizontal lines (Blakemore and Cooper, 1970). When the kittens were moved to a more visually varied environment, they were poor at discriminating

vertical lines. Thus it seems that the visual system learns which kernels to use and PCA serves as a simple model for how it does that.

2.3.1 PCA of image patches

As an example, we will return to the image of Mona Lisa in Figure 2.2 and use PCA to learn kernels at a scale of 10 that will allow us to encode most of the variance in the image. First, we will sample 10-by-10 image patches from the image. We can do this at random or we can split the image into non-overlapping patches. We will then flatten the 10-by-10 patches so that they become 100-dimensional vectors. For each pixel, we must also subtract the mean across images, so that the vectors will have a mean of zero across images. Then we can perform PCA, which will give us the matrices $\mathbf{A}$ and $\mathbf{W}$.

Recall that the principal components, the columns of $\mathbf{W}$, are vectors in the original data space, here of 100 dimensions corresponding to flattened 10-by-10 image patches. We can thus visualise the principal components as image patches by unflattening them. Figure 2.4 shows the first 6 principal components as image patches and allow us to interpret the components in the image that the principal components capture. The first principal component is an almost uniform patch. This is the component that captures the overall luminance of the patch. The second principal component has positive values at the top and negative values at the bottom. It will capture a horizontal edge dividing the patch. Similarly, the third principal component will capture vertical edges. The fourth principal component is particularly interesting because it resembles the kernels that came from the lateral inhibition network in Section 2.2: it represents contrast between the center and the periphery of the patch. Finally, note that the sixth patch has a more complex structure representing horizontal lines.

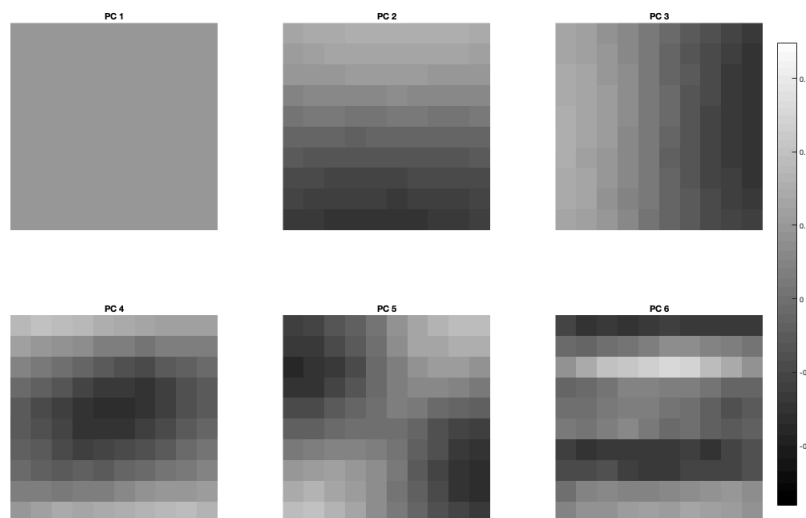


Figure 2.4: The first 6 principal components learned from 10-by-10 image patches taken from Mona Lisa. Note that the patches are shown on the same color scale shown in the color bar to the right..

We can see the effect of encoding the image patches using $L < N$ principal components by reconstructing the image patches using only L principal components. We do this using Equation 2.8 selecting only the first L columns of matrices $\mathbf{A}$ and $\mathbf{W}$. Note that we must also remember that before PCA we subtracted the mean of $\mathbf{J}$ from the rows of $\mathbf{J}$, so we will have to add it back to the reconstruction. Finally we have to unflatten the 100-dimensional vector so that they become 10-by-10 pixel image patches.

Figure 2.5 illustrates the reconstruction of a set of image patches that form a detail from the original image in the left panel of Figure 2.2. In the center panel, the patches are reconstructed using the first

principal components. Although the image is more blurry, the reconstruction captures a large part of the variance in the original shown in the left panel. This shows that PCA has managed to compress a large part of the variance in the image onto only six components. With a weight matrix determined by PCA, a linear neural network would thus be able to compress the dimension of the visual input substantially with a modest loss of information. Finally, note that in the right panel, the patches have been reconstructed using only the first principal component. This image is a pixelated version of the original. This is because all the image patches are represented by only one component, which is, as we note above, a uniform patch. The image patches are therefore all uniform varying only in intensity, which is what gives the image its pixelated appearance.



Figure 2.5: Left: A detail from the original image Mona Lisa in Figure 2.2. Center: The same detail made of image patches reconstructed using six principal components. Right: The same detail reconstructed using only one principal component. See main text for details..

2.3.2 Exercise - Learning image features using Principal Component Analysis

Load the image of Mona Lisa that you find in the exercise folder and transform it to a set of non-overlapping patches each 10-by-10 pixels. Flatten the patches into 100-dimensional vectors. Stack the vectors into a matrix, $\mathbf{J}$, where each row is an observation, in our case an image patch, and each column correspond to a feature, in our case a pixel. Use PCA to recreate Figures 2.4-2.5. You do not have to select the exact same detail of Mona Lisa that was selected in Figure 2.4. Repeat the process for other image patch sizes and other images. Compare your image reconstructions and your principal components across image patch sizes and other images to see if you get similar results.

Bayesian models of perception

3.1 Introduction

Perception often seems effortless. When we recognise the face of a friend it seems to happen easily, quickly and automatically. This can lure us into believing that perception is a simple process and that our perceptual experience is merely a reflection of the world as it is. This belief is far from the truth. Yet, an anecdote from the early years of AI research tells that this false belief lured pioneering researchers. In 1966 Seymour Papert set up a summer project for students at MIT. The goal of the project was to detect and identify objects from images. It took several decades for the field to reach useful solutions and the problem has still not been completely solved, so Papert seem to have underestimated the challenge.

From the earliest studies of human perception, it has become clear that perception is an extremely complex process. From the most recent studies we learn that we still do not fully understand it. What we do know is that perception depends not only on sensory information but also on the state of the observer, an understanding of the world around us and on perceptual biases.

Perception can be thought of as the brain's solution to an inverse problem: estimate the state of the world given sensory information. This type of problem is quite similar to the scientific endeavour of finding the state of the world given measurements. In both perception and in science such inverse problems are typically ill-posed or under-determined. This means that there is not enough data to find a unique solution. Yet, perception and science both solve this type of problems by adding information. This information comes in the form of models of how the world works and prior knowledge of the state of the world.

We can think of inverse problems in terms of Bayes' rule. Assume that the data can, in general, be represented by a vector, $\mathbf{d}$. The data could be the activation level of all the photo-receptors in the eyes, or it could be measurements in a scientific experiment. Likewise, assume that the state of the world can also be quantified by a vector, $\mathbf{w}$. Then Bayes' rule can be written as

$$P(\mathbf{w} | \mathbf{d}) = \frac{P(\mathbf{d} | \mathbf{w})P(\mathbf{w})}{P(\mathbf{d})} \quad (3.1)$$

The problem of perception is to determine the posterior probability of the state of world given sensory information, or data, $\mathbf{d}$, and to determine the most probable state(s) of the world based on this posterior distribution. According to Bayes' rule in Equation 3.1, the posterior probability is proportional to the likelihood, $P(\mathbf{d} | \mathbf{w})$, and the prior probability, $P(\mathbf{w})$.

Typically, we are not interested in the state of the entire world but only a small part of the world. For example, we might be interested in whether there is a tiger hiding in the bushes. In this example, the sensory data could be the sound coming from the bushes: Does it contain the sound of a tiger's low rumble or not? The likelihood would then the probability of the particular sound coming from the bushes *given* that there is a tiger hiding there. Tigers sometimes rumble and sometimes do not. In order to determine the likelihood, we need to estimate the probability that they do, or, even better, that this particular, albeit hypothetical, tiger rumbles. The likelihood thus contains a probabilistic model of how the world works, or more precisely, how it generates sensory data.

The prior probability is the probability of the state of the world prior to any sensory input as opposed to the posterior probability, which can be determined only after sensory input. It could be the probability of a tiger hiding in the bushes in general and would be based on previous experience with tigers and bushes.

The denominator in Bayes' rule, $P(\mathbf{d})$ is sometimes referred to as the evidence. For some purposes, this term can be ignored. To see why, let us rephrase the problem of determining whether there is a tiger hiding in the bushes as an inequality

$$P(\text{tiger} \mid \text{rumble}) > P(\text{no tiger} \mid \text{rumble})$$

This inequality implements the *maximum a posteriori* (MAP) decision rule: we decide that there is a tiger if this is the more probable outcome. If we insert Bayes' rule on both sides, we get

$$\frac{P(\text{rumble} \mid \text{tiger})P(\text{tiger})}{P(\text{rumble})} > \frac{P(\text{rumble} \mid \text{no tiger})P(\text{no tiger})}{P(\text{rumble})}$$

Note that the evidence, $P(\text{rumble})$, is the only term that does not depend on the state of the world. It is therefore the same on both sides of the MAP decision rule inequality. We can thus multiply the by $P(\text{rumble})$ on both sides and remove it from the inequality. In effect, we may ignore the evidence term when making decisions by comparing the posterior probabilities of various outcomes.

Another way to understand the role of the evidence is to rewrite it using the law of total probability so that Bayes' rule can be written in the form

$$P(\mathbf{w} \mid \mathbf{d}) = \frac{P(\mathbf{d} \mid \mathbf{w})P(\mathbf{w})}{\sum_{\mathbf{w}} P(\mathbf{d} \mid \mathbf{w})P(\mathbf{w})}$$

In this form, it is clearer that the evidence in the denominator can be thought of as a normalisation term ensuring that $\sum_{\mathbf{w}} P(\mathbf{w} \mid \mathbf{d}) = 1$, which must hold for any discrete probability distribution, in general.

3.2 The use of prior information in visual perception

3.2.1 The Necker cube problem

Depth perception, our ability to perceive a three-dimensional world from two-dimensional images, is a good example of how the visual systems seems to solve ill-posed problems in which there are fewer equations than unknown variables. Although binocular vision is important for depth perception, the visual system uses many other cues to determine distance. A good demonstration of this is that we can still perceive depth with one only one eye. Another good demonstration is the Necker cube illustrated in panel (A) of Figure 3.1. We perceive this two-dimensional drawing as a three-dimensional wire-frame structure. In solving this problem, the visual system thus seems to go beyond the sensory information given, although it still does not quite manage to solve the problem entirely, as it fails to find a unique solution. Instead it finds two possible interpretations illustrated in panels B-C of Figure 3.1. Strangely, the visual system's interpretation of the Necker cube seems to alternate between two solutions causing what is know as a bistable percept.

Analysing the Necker cube problem in terms of Bayesian inference is a good way to illustrate how the framework can be applied in general. The analysis will, however, rely on a number of simplifying assumptions. The first assumption is that we can specify the two-dimensional drawing of the Necker cube and the underlying three-dimensional structure in terms of the eight vertices only. Hence, the data, $\mathbf{d}$, is a set of eight two-dimensional coordinates stacked into vector form, and the world, $\mathbf{w}$, is a set of eight three-dimensional coordinates also stacked into vector form.

For this problem, the data, $\mathbf{d}$, and the world, $\mathbf{w}$, are continuous variables unlike the rumbling tiger problem described above in which the data, $\mathbf{d}$, and the world, $\mathbf{w}$, consisted of discrete states. We therefore write Bayes' rule as

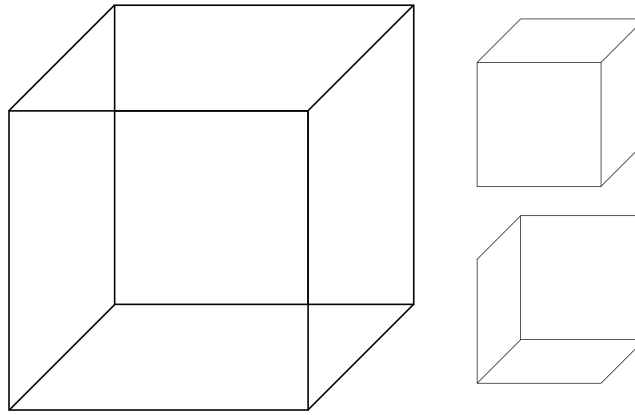


Figure 3.1: The Necker cube (Left) appears to be a three-dimensional wire-frame cube. The three-dimensional percept is bistable in that the cube can be perceived as two distinct three-dimensional structures (Top and bottom right). Figure by user Stevo-88, Wikimedia Commons..

$$p(\mathbf{w} | \mathbf{d}) = \frac{p(\mathbf{d} | \mathbf{w})p(\mathbf{w})}{p(\mathbf{d})} = \frac{p(\mathbf{d} | \mathbf{w})p(\mathbf{w})}{\int_{\mathbf{w}} p(\mathbf{d} | \mathbf{w})p(\mathbf{w})} \quad (3.2)$$

where the use of lower case p means that the term is a probability *density*.

As in the the rumbling tiger problem described above, the task of the observer is not to characterise the posterior probability distribution fully but only to find the most probable estimate(s) of the world, the *maximum a posteriori* (MAP) solution, $\mathbf{w}_{map}$, given the sensory data $\mathbf{d}$.

$$\mathbf{w}_{map} = \arg \max_{\mathbf{w}} p(\mathbf{w} | \mathbf{d}) = \arg \max_{\mathbf{w}} p(\mathbf{d} | \mathbf{w}) P(\mathbf{w})$$

Here we have inserted Bayes' rule and noticed that since the evidence term, $p(\mathbf{d})$, in the denominator does not depend on the world, w , can be ignored as we can multiply by positive terms that do not depend on the world with no effect on the MAP solution.

3.2.2 The Uninformative prior and the Maximum Likelihood Estimate

In the case of a non-informative, or *flat*, prior, all worlds are equally probable *a priori* so that $p(\mathbf{w})$ is a constant for all $\mathbf{w}$. In this case we can disregard the prior as it will not influence the solution. This solution is thus based only on the likelihood and is often referred to as the *maximum likelihood estimate* (MLE).

$$\mathbf{w}_{mle} = \arg \max_{\mathbf{w}} p(\mathbf{d} | \mathbf{w})$$

The likelihood term describes how the two-dimensional image is generated. There are two steps in the process. In the first step, the three-dimensional world, $\mathbf{w}$, is projected onto the two-dimensional image plane. We will refer to the projection of each three-dimensional coordinate as μ_i . In the second step, Gaussian sensory noise is added to the data, so that each data point d_i is distributed as $d_i \sim \mathcal{N}(\mu_i, \sigma)$ where $\mu_i = \{x_1, \dots, x_N, y_1, \dots, y_N\}$ is the set of the N x - and y -coordinates of the projection. We assume that the noise is *independent and identically distributed* (IID) across points, which means that the covariance is zero and the variance, σ^2 , of the noise is the same for all points. We can now write the likelihood function as

$$p(\mathbf{d} | \mathbf{w}) = \prod_{n=1}^N f(d_n, \mu_n, \sigma)$$

where f is the Gaussian probability density function (pdf)

$$f(d_n, \mu_n, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{d_n - \mu_n}{\sigma}\right)^2}$$

Working with the logarithm of the likelihood, the *log likelihood*, provides some simplification and also numerical stability. Note that this does not alter the MAP solution because the logarithm is a monotonically increasing function.

$$\mathbf{w}_{mle} = \arg \max_{\mathbf{w}} P(\mathbf{w} | \mathbf{d}) = \arg \max_{\mathbf{w}} \log P(\mathbf{w} | \mathbf{d})$$

Since the logarithm of the Gaussian pdf is

$$\log f(d_n, \mu_n, \sigma) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2} \left(\frac{(d_n - \mu_n)^2}{\sigma^2} \right)$$

the logarithm of the likelihood is

$$\log P(\mathbf{d} | \mathbf{w}) = \sum_{n=1}^N \log f(d_n, \mu_n, \sigma) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^N (d_n - \mu_n)^2$$

We can subtract and divide out any positive terms that do not depend on the world, $\mathbf{w}$, when we search for the $\mathbf{w}_{mle}$ solution. Here, only the projection, μ , of the world, $\mathbf{w}$, onto the image plane will depend on the world. Also, instead of maximising the logarithm of the likelihood we can minimise the negative of the logarithm of the likelihood.

$$\mathbf{w}_{mle} = \arg \min_{\mathbf{w}} (-\log p(\mathbf{d} | \mathbf{w})) = \arg \min_{\mathbf{w}} \sum_{n=1}^N (d_n - \mu_n)^2$$

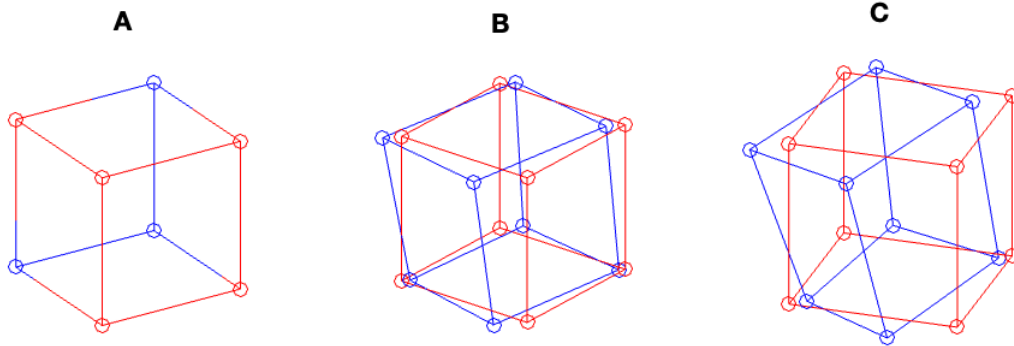


Figure 3.2: The Necker cube (A) as in Figure 3.1. The true three-dimensional structure is drawn in red while the MLE solution is drawn in blue. Viewed in the projection plane the solution and the true structure seems identical. When viewed from an angle shifted 20° (B) or 40° in the azimuthal plane the two structures no longer overlap..

An MLE solution can thus be found simply by minimising the sum of squared errors between the sensory data, $\mathbf{d}$ and the projection, $\boldsymbol{\mu}$, of the world, $\mathbf{w}$ onto the image plane. Note that the solution is not unique. We can easily see that by assuming that the image plane is the z -plane (and that the size of the sensory image is small compared to the size of the eye). This assumption does not incur any loss of generality as we are free to choose our coordinate system so that the z -plane aligns with the image plane. In this case the x - and y -coordinates of the projection are simply the x - and y -coordinates of the three-dimensional points. There are no constraints on the z -coordinate, which can take on any value. MLE solutions will therefore appear as perfectly good solutions when viewed from the image plane but will generally prove to be poor solutions when viewed from other planes. This is illustrated in Figure 3.2

3.2.3 The informative prior and the Maximum Posterior Solution

We are now ready specify a prior on the world, $\mathbf{w}$. The prior probability density, $p(\mathbf{w})$, is a probability density over all possible three-dimensional shapes with eight vertices. We could specify that as a 24-dimensional probability density over all possible sets of eight three-dimensional points. We can, however, also specify the prior on other attributes, of the three-dimensional shape. For illustrative purposes, we will specify a prior probability density on the 24 internal angles, $\theta_1, \dots, \theta_M$ of the structure, so that . This prior is a prior towards angles of 90° , i.e. away from flat and pointy structures, which would have angles closer to 0° or 180° . We can implement this prior using a probability density that is maximal at 90° , symmetric, and minimal at 0° and 180° . We could use the von Mises distribution, which is defined on the circle but, we will, for simplicity, use a Gaussian distribution centered at $\mu_p = 90$ since it is a reasonable approximation to the von Mises distribution when the standard deviation, σ , is small. We will also assume that the density over the angles are IID so that the prior can be written as

$$p(\boldsymbol{\theta}) = \prod_{m=1}^M f(\theta_m, \mu_p, \sigma_p)$$

Recall that we only need the numerator of the posterior when we seek the w_{map} solution. We can now express the logarithm of the numerator as

$$\begin{aligned} \log((\mathbf{d} | \mathbf{w}) p(\boldsymbol{\theta})) &= \log p(\mathbf{d} | \mathbf{w}) + \log p(\boldsymbol{\theta}) \\ &= -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^N (d_n - \mu_n)^2 - \frac{M}{2} \log(2\pi\sigma_p^2) - \frac{1}{2\sigma_p^2} \sum_{m=1}^M (\theta_m - \mu_p)^2 \end{aligned}$$

When we seek the MAP solution we can subtract and divide by all terms that do not depend on the world, $\mathbf{w}$, so that we arrive at

$$\begin{aligned} w_{map} &= \arg \min_{\mathbf{w}} (-\log p(\mathbf{w} | \mathbf{d})) \\ &= \arg \min_{\mathbf{w}} \left(\frac{1}{\sigma^2} \sum_{n=1}^N (d_n - \mu_n)^2 + \frac{1}{\sigma_p^2} \sum_{m=1}^M (\theta_m - \mu_p)^2 \right) \end{aligned}$$

We can thus find the MAP estimate by minimising a *weighted* sum of squares in which the likelihood terms are weighted by the *reliability*, $r = \sigma^{-2}$, and the prior terms are weighted by the reliability, $r_p = \sigma_p^{-2}$. Note that if the variance of the sensory noise, σ^2 is large then the reliability, r , of the sensory information is small and the likelihood term, which pertains to sensory information will, accordingly have a less influence on the MAP estimate, w_{mle} . This would correspond to the case where the observer caught only a fleeting glimpse of the object and would therefore rely more strongly on prior assumptions. The reliability, r_p , of the prior is not related to noise but to how informative or *strong* the prior is. If the reliability of the prior is very small, then the prior distribution would be near flat, or uninformative.

The observer would therefore have to rely on sensory information. Conversely, if the reliability of the prior is high, then the prior distribution is narrowly centered around the mean so that estimates of the state of the world that deviates only slightly from the observer's prior assumptions are deemed highly improbable. The trade-off between sensory and prior information can be captured by the ratio $\frac{r_p}{r}$ of the two reliabilities allowing us to express the MAP estimate as

$$\mathbf{w}_{map} = \arg \min_{\mathbf{w}} \left(\sum_{n=1}^N (d_n - \mu_n)^2 + \frac{r_p}{r} \sum_{m=1}^M (\theta_m - \mu_p)^2 \right)$$

Note that we can define a prior on any feature of the three-dimensional world. We could, for instance, define a 90° prior on the 24 angles of the structure, so that $a_m = 90^\circ$ for all m . We also need to define a value for $\frac{r_p}{r}$. If the value chosen is too small, then the prior term will have little effect on the MAP solution, which therefore will be similar to the MLE solution shown in Figure 3.2. If it the value chose is too large, then the prior term will dominate, which results in solutions with angles of 90° as dictated by the prior but at the cost of ignoring the sensory data, d , in the likelihood term prior. Therefore, their projection images bear little resemblance to the projection of the true structure from any angle. This is illustrated in Figure 3.3. For appropriate values, the MAP solution can be a perfect fit so that its projection image overlaps with the projection image of the true structure from any viewing angle. In this case, the MAP solution for the three-dimensional structure is consistent with what the three-dimensional structure that humans perceive from the two-dimensional image.

A prior must be based on prior experience and cannot be based on the (sensory) data. Choosing a 90° prior for a cube only serves to demonstrate the effect of prior information. It is unlikely that the efficacy of the 90° prior will generalise to other three-dimensional structures such as the hexagonal prism in Figure 3.1. However, an open angle prior, centered at around 100° , works for both the Necker cube and the hexagonal prism, for appropriate values of $\frac{r_p}{r}$. This indicates that such a prior may play a role in visual perception.

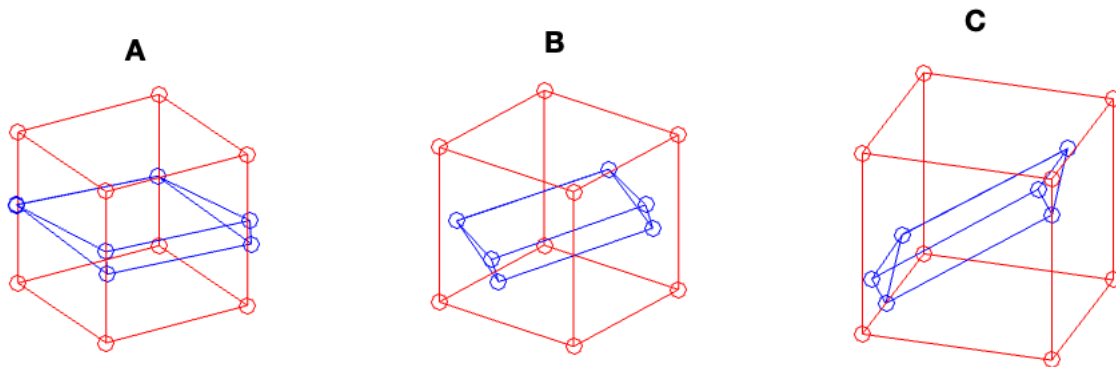


Figure 3.3: The Necker cube (A) as in Figures 3.1. The true three-dimensional structure is drawn in red while the MAP solution is drawn in blue. Here the value of $\frac{r_p}{r}$ is too large so that the prior term dominates and sensory information is ignored. Therefore, although the internal angles of the solution are very close to 90° , as dictated by the prior, the projection of the MAP solution appears to not overlap with the the projection of the true structure regardless of the viewing angle..

3.2.4 The light from above prior

Shading provides important cues to depth perception. This is illustrated in the two panels of Figure 3.4, which our visual system interprets as three-dimensional structures. The panel to the left shows a number of objects that appear to be hemispheric protrusions except for one object that appears to be a hemispheric cavity. Contrary, the panel to the right shows a number of objects that appear to be hemispheric cavities except for one object that appears to be a hemispheric protrusion. The two figures are, in fact, identical except for a 180° rotation. You can try to view the figure upside-down: The objects that appeared to be protrusions when viewed upright now appear to be cavities and vice versa. This raises the question of why the orientation in the viewing plane so greatly affects the visual system's interpretation of depth.

The three-dimensional interpretation of the objects in Figure 3.4 may be influenced by the *light from above prior* meaning that the visual system assumes that the light source is located above the object. Evolutionary, this seems like a reasonable prior assumption in that the main source of light during most of evolution has been the sun, which is indeed located above any object. Assuming that light comes from above agrees with the interpretation of a protrusion being bright at its top because it is lit from above and dark at its bottom because of its own casting shadow. Conversely, a cavity is bright only at the bottom where light can shine through the cavity but dark at the top, which lies in a shadow. Note that objects that appear as protrusions might as well be cavities lit from below. Although we can consciously convince ourselves that this is a reasonable interpretation, we cannot *see* it because we cannot persuade our visual system of this interpretation.

3.2.5 Exercise - The Necker cube

In this exercise we will solve the Necker cube problem using the Bayesian approach described in Sections 3.2.1-3.2.3. The problem is to infer the world, $\mathbf{w}$, which is a three-dimensional wire-frame structure from the data, $\mathbf{d}$, which is a two-dimensional image. The Bayesian approach is to maximize the posterior probability $p(\mathbf{w} | \mathbf{d})$. For practical reason, we will minimize the negative logarithm of the posterior probability instead as described in Sections 3.2.2-3.2.3. First, we will assume an uninformative prior and let the squared error correspond to the likelihood function as described in Section 3.2.2.

We will use Matlab in this exercise. You must place all the Matlab files that come with the exercise in your active directory. Open the Matlab function `negLogPosterior.m`. You only have to fill out the argument of the sum of squares that will calculate the negative log posterior using the variables $\mathbf{w}$ and $\mathbf{d}$. The variable $\mathbf{w}$ is a 3-by-8 matrix of the estimate of the three-dimensional world. The variable $\mathbf{d}$ is a 2-by-8 matrix of the perceived image. Each column in both $\mathbf{w}$ and $\mathbf{d}$ holds the coordinates of one corner. You can also use the function `Project(w,M)` to calculate the projection of the three-dimensional world onto the two-dimensional viewing plane. The variable $\mathbf{M}$ is a 2-by-4 projection matrix. Once you have you have completed the `negLogPosterior.m`, you can run the `NeckerExercise.m` function and it will show a plot of your solution and the true cube.

Inspect the figure visually. View it from different angles by clicking and dragging the figure. Describe the solution you get. Is it a correct solution? There are, at least, two ways that it can be the wrong solution. What are they? Note that the solution you get can depend on the random initialisation, so you might want to try more than one solution.

Next, try using an informative prior. Use the calculation of the angles (in degrees) of the three-dimensional structure calculated in the variable `Angles` (a prior for right angles will work well for the cube). Again, inspect the figure visually. View it from different angles. Try this for different values of $\frac{\tau_p}{\tau}$ as described in Equation 3.2.3. Describe the solutions you get when the value is too low, too high and just right. Which value seems just right?

Do the same thing with a hexagon cylinder to test whether the prior you came up with for the cube also works for a different structure. Open the Matlab function `HexagonalExercise.m`. It works just like `NeckerExercise.m` function. First try the uninformative prior that you used to solve the Necker cube problem. Just as for the Necker cube problem you can do this by editing the `negLogPosterior.m` file. The results should be similar to the results for the Necker cube problem. Then try the prior that you

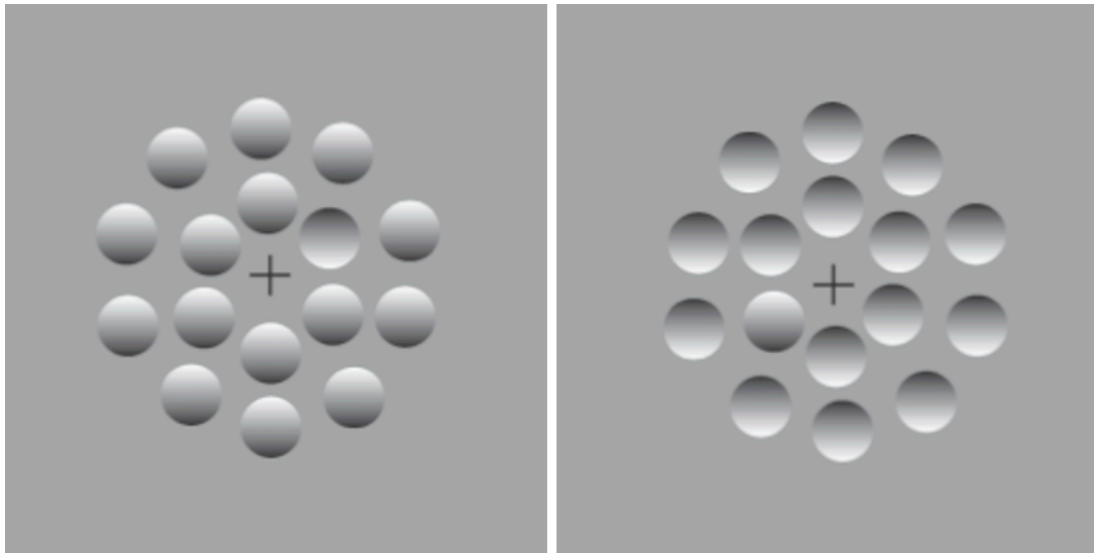


Figure 3.4: Our visual system use shading cues to interpret the above figures as three-dimensional. The panel to the left show a number of objects that appear to be hemispheric protrusions except for one object that appears to be a hemispheric cavity. Contrary, the panel to the right show a number of objects that appear to be hemispheric cavities except for one object that appears to be a hemispheric protrusion. The two figures are, in fact, identical except for a 180° rotation. This shows that the visual system's interpretation of the three-dimensional structure depends on the orientation of the object in the viewing plane. This interpretation may be due to the visual system relying on a prior assumption of light coming from above. Figure © by Champion and Adams (2007), Journal of Vision..

used for the Necker cube problem. Describe the results. Try to find a prior that works for the hexagon problem. Describe your approach. Try finding a prior that works for both the Necker cube and the Hexagon. Describe your approach.

3.3 The use of marginalisation in perception

3.3.1 Depth from shading

So far, we have learned how our visual system use prior assumption to interpret sensory information. To demonstrate this we have focused on the visual system's ability to perceive depth in two-dimensional figures. In this section we will study *marginalisation* as another method that the visual system may use to interpret sensory information. To demonstrate this, we will continue focusing on the visual system's ability to perceive depth from two-dimensional figures.

First, we will revisit Figure 3.4. The light from above prior seemed a reasonable explanation for how the visual system can distinguish between hemispheric protrusions and hemispheric cavities, but recall from Section 3.2 that monocular depth perception is a highly ill-posed problem in that there is an unknown variable for every point in the object. In the case of the Necker cube, which we parameterized using the eight vertices, we had eight unknowns. In order to find a reasonable MAP solution, we used a high-dimensional prior. This did not solve the problem completely since the Necker cube remains a bistable percept. How many points do we need to specify that an object is a hemispheric protrusion or cavity? Perhaps a four-by-four grid of three-dimensional points would suffice. In that case we would have 16 unknowns and it should be a bit surprising that even without using prior information on the

orientation of the light source, we can only see the object as a hemispheric protrusion or cavity when in fact there should be infinitely many three-dimensional objects that could create the sensory image. This is illustrated in Figure 3.5 where Panel A shows an image that appears to show a hemispheric protrusion lit from the left or a hemispheric cavity lit from the right. These two solutions are depicted in Panel B (1 and 5) along with three other solutions (2-4) that are not hemispheric shapes, yet they will generate the sensory image when lit from the direction indicated by the arrow next to the object. Why does our visual system not offer these structures as possible causes of the sensory image?

We can phrase the problem of inferring depth from shading in terms of Bayes' rule by splitting the world, $\mathbf{w}$, into two components that influence the sensory data: the shape of the object, which we will here denote as $\mathbf{w}_1$ and the orientation of the light source, which we will denote as $\mathbf{w}_2$. Then, Bayes' rule can be written as

$$p(\mathbf{w}_1, \mathbf{w}_2 \mid \mathbf{d}) = \frac{p(\mathbf{d} \mid \mathbf{w}_1, \mathbf{w}_2)p(\mathbf{w}_1, \mathbf{w}_2)}{p(\mathbf{d})} \quad (3.3)$$

Since the posterior has two components, as in Equation 3.3, the most probable shape (the MAP estimate of the shape) depends on the orientation of the light source. One shape might be the most probable shape given a certain lighting angle, another shape will be the most probable given another lighting angle, so we might not be able to find a single best solution. What we are really interested in is estimating the posterior probability $p(\mathbf{w}_1 \mid \mathbf{d})$ and we can do this by integrating or, *marginalising*, over the lighting angle, $\mathbf{w}_2$

$$p(\mathbf{w}_1 \mid \mathbf{d}) = \int_{\mathbf{w}_2} p(\mathbf{w}_1, \mathbf{w}_2 \mid \mathbf{d}) \quad (3.4)$$

Equation 3.4 tells us that the most probable shape, is the shape that is most probable *across* all viewing angles, $\mathbf{w}_2$. Marginalising over the viewing angle, $\mathbf{w}_2$, can help us find this solution even when the prior is uninformative, meaning that all shapes and viewing angles are equally probable *a priori*. In that case we can insert Bayes' rule from Equation 3.3 into Equation 3.4 to get

$$p(\mathbf{w}_1 \mid \mathbf{d}) = \int_{\mathbf{w}_2} p(\mathbf{w}_1, \mathbf{w}_2 \mid \mathbf{d}) = \int_{\mathbf{w}_2} \frac{p(\mathbf{d} \mid \mathbf{w}_1, \mathbf{w}_2)}{p(\mathbf{d})} \quad (3.5)$$

Equation 3.5 tells us that the most probable shape is the shape that would create the sensory image even when seen across all viewing angles. Panel C shows five sensory images that a shape would generate across a small range of lighting angles, shown as arrows below the sensory images. Note how the sensory image varies far very much away from the sensory image in Panel A and in the center sensory image in Panel C. Hence, although the likelihood of that shape would be high for one particular lighting angle, it would be quite low for other lighting angles, so the integral in Equation 3.4 would take on a fairly low value. This might be the reason for why the visual system concludes that this shape is improbable. Compare these observations to the sensory images that the hemispheric protrusion in Panel D generates across varying lighting angles. These sensory images are all quite similar to the sensory image in Panel A. Hence, the likelihood of that shape would be high across a broader range of lighting angles, so the integral in Equation 3.4 would take on a fairly high value. This might be the reason for why the visual system concludes that this shape is more probable.

The marginalisation approach described here is similar to the use of the *Bayes' factor* for model evaluation in machine learning. The Bayes factor is the ratio of the marginal likelihoods of two models, M_a and M_b

$$\frac{P(D \mid M_a)}{P(D \mid M_b)} \quad (3.6)$$

A Bayes factor much greater than 1 indicates that model M_a is better whereas a Bayes' factor much less than 1 indicates that model M_b is better. Note that the Bayes factor is different from the maximum likelihood estimate that we previously encountered because the likelihood terms in Equation 3.6 are

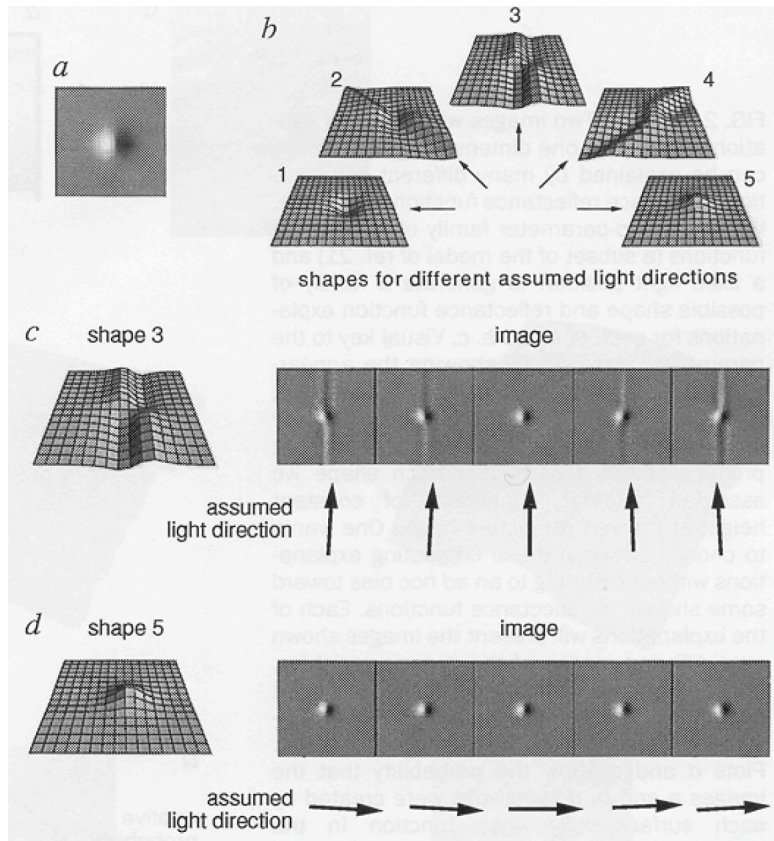


Figure 3.5: Our visual system interprets the sensory image in (a) as a protrusion lit from the left or a cavity lit from the right. All the objects in (b) will, however, produce the same sensory image when lit from the direction indicated by the arrows. When the object (shape 3) in (c) is lit from a narrow range of angles it produces sensory images very unlike the sensory image in (a). Therefore, it is not a good solution when evaluated by marginalising over the lighting angle. Contrary the object (shape 5) in (d) produces sensory images very similar to the sensory image in (a) across the same range of lighting angles. Therefore, it is a good solution when evaluated by marginalising over the lighting angle. Figure © by Freeman (1994), Nature.

marginalised across the model parameters. If models M_a and M_b are parameterised by parameter vectors θ_a and θ_b then the Bayes factor can be written as

$$\frac{P(D | M_a)}{P(D | M_b)} = \frac{\int_{\theta_a} P(D | \theta_a, M_a) P(\theta_a | M_a)}{\int_{\theta_b} P(D | \theta_b, M_b) P(\theta_b | M_b)} \quad (3.7)$$

In practice, these integrals are typically very complex and must be estimated numerically, typically using Markov Chain Monte Carlo methods if precise estimates are needed. The visual system could, however, use coarse approximations by summing over only a few selected values.

Comparing the Bayes factor approach with the marginalisation approach described above, note that the models, M_a and M_b would be models of the world, w , each consisting of the lighting angle and the three-dimensional structure. Whereas the Bayes' factors integration across all parameters only makes sense for two models that are parameterised differently, the marginalisation approach described here distinguish between two models that are parameterised in the same way and therefore only integrates across the subset of parameters that are irrelevant. These are called *nuisance parameters* and would the

lighting angle in the example above. Common to the marginalisation approach and the Bayes' factor is that they seek solutions that are robust to variations in the parameters.

3.3.2 Impossible triangle

The *impossible triangle* illustrated in the lower left of Figure 3.6 serves as another example of how the visual system might use a marginalisation approach. Only in this case, the object has been carefully designed to trick the marginalisation approach to create an illusory percept.

Our visual system interprets the impossible triangle as a closed triangular shape but our conscious reasoning soon tells us that this is impossible. The figure is however a photograph and the object is very much real. The objects' mirror reflection shows us the object's true three-dimensional structure: it is not a closed triangular shape. Our visual system has come up with a wrong interpretation of the object as a closed triangular shape is wrong, but why?

As in the problem of inferring depth from shading, we will again phrase the problem in terms of Bayes' rule by splitting the world, w , into two components, w_1 and w_2 , so that Bayes' rule is given by Equation 3.3. Again, w_1 denotes the three-dimensional structure of the object, but now, w_2 denotes the viewing angle. If the visual system uses the marginalisation approach described by Equation 3.5 it should find an object that will produce the sensory image of a closed triangular shape when viewed across a wide range of angles rather than an object that will produce the sensory image of a closed triangular shape only when viewed from a narrow range of angle. Hence the solution of the visual system is a closed triangular shape: Even though it is impossible, it would indeed produce the sensory image across a wide range of viewing angles. In contrast, the real object revealed by the mirror reflection in Figure 3.6 will produce a sensory image of a closed triangular shape only when viewed from a narrow range of angles. This is called the *generic viewpoint assumption*. The visual system assumes that the viewing angle was generic rather than very particular.



Figure 3.6: The object in the lower left, appears to be an "impossible triangle", an illusory object that appears to be closed triangular from but violates the laws of geometry. Although our visual system interprets the object in this way, our conscious reasoning can deduce that it is impossible for this object to exist. The figure is, however, a photograph of a real object. The real three-dimensional structure is revealed by the objects mirror reflection showing that the figure is not actually a closed triangle contrary to what we perceive when viewing the figure from a particular angle. Photograph © by Bruno Ernst.

3.3.3 Exercise - The use of marginalisation in visual perception

Solve the simple perceptual task facing a monkey in the jungle using a Bayesian approach. Spell out which terms are posteriors, priors, likelihoods, what terms are being discounted. You can assume that the reflection spectres follow Gaussian (normal) distributions.

1. Out in the jungle, 15% of the juju-fruits are ripe. Ripe juju-fruits are orange, on average reflecting light with a wavelength of about 600 nm with some variation (standard deviation of 50 nm). Unripe juju-fruits are green with a wavelength of 500 nm (standard deviation 50 nm). What is the probability of a juju-fruit reflecting light with a wavelength between 540-550 nm is ripe? Note that you can calculate the probability of a fruit having a wavelength, w , smaller than some value using the cumulative Gaussian distribution function (`norm.cdf` in Numpy or `normcdf` in Matlab).
2. Only 10% of the fruits in the jungle are juju-fruits. 50% are mongo berries. 80% of the mongo berries are ripe. When ripe, mongo berries reflect light with a wavelength of 580 nm (standard deviation 20). When unripe, they reflect light with a wavelength of 520 nm (standard deviation 20). The remaining fruits are all chakavas. Only 10% of the chakavas are ripe. When they are ripe, they reflect light with a wavelength of 400 nm (standard deviation 100). When they are unripe, they reflect light with a wavelength of 550 nm (standard deviation 100). What is the probability that a random fruit is ripe if it reflects light with a wavelength between 540-550 nm?

CHAPTER 4

Human category learning

In Section 1.4 we noticed that there are two types of information that humans and machines can store. In Sections 1.5 -1.6 we studied how humans store specific sequences of items. This corresponds to rote memorisation, i.e. veridical storage of training data. We noticed that this type of information is trivial to accomplish in computers. All that is needed is to encode the information into binary format and to be able to decode it at a later stage. These encoding and decoding algorithms cannot be considered artificial intelligence as they are, in general, pre-specified and not adaptive. We will now proceed from the storage of information to learning. In particular, we will study category learning in humans and machines.

Category learning is a common form of learning for both humans and machines. We can define category learning as acquiring the ability to apply labels to inputs. In machine learning, category learning can be supervised, when learning a function that maps inputs (features) to outputs (labels), or unsupervised, when learning a mapping of inputs to structures in the data, like clusters. These forms of feature-based learning also occur in humans. One example could be the ability to categorise an animal as a cat. Children quickly learn this by seeing only a few examples of cats. They may also need a few examples of animals that are not cats to understand that not all animals are cats. Based on such observations, two theories for how categories are represented in human memory have been developed. The *exemplar* theory states that the category is represented as a collection of *exemplars*, stored as points in a multi-dimensional feature space. According to this theory, classification of new, unseen, inputs is based on a comparison between the input and the exemplars stored in memory. The new input is simply categorised in the same category as the most similar exemplar stored in memory. If, for example, the animal that child sees resembles a previously seen cat more than it resembles any other animal, then the animal is categorised as a cat. Although this seems like a good approach during the early stages of learning, when the number of exemplars stored in memory is low, this approach seems unfeasible in later stages of learning, when that number increases. Imagine that you would have to store memories of all the animals you have ever encountered in order to be able to recognise a cat. At this stage, it seems more reasonable that you have formed an abstract idea of what a cat is. The *prototype-based* theory states that the category is represented as a *prototype*. A prototype can be formed by taking the average of previously seen exemplars belonging to the category. For example, the prototypical cat, would be the average cat, based on your experience.

The prototype distortion task has been influential in the study of how humans learn and represent categories. In this task, a prototype is selected as a random point in feature space. Feature vectors with a known Euclidian distance, d , to the prototype are then sampled. These feature vectors are transformed into actual combinations of features, which are typically represented visually. One commonly used feature set is the x - and y -coordinates of a set of dots. With this feature set, a six-dimensional feature vector is presented visually as three dots on a screen with the x - and y -coordinates of the dots determined by the feature vector. In the learning phase of the prototype distortion task, two training sets are presented to the observer. One set has a relatively small distance, d_- , to the prototype and the test participants are informed that these are all examples of members of the category. Another set has a larger distance, d_+ , to the prototype and the test participants are informed that these are all examples of non-members of the category.

Once the training phase is over, new examples are generated and the test participant is asked to categorise these. Based on the test-participants responses, we can test the predictions of the prototype and exemplar theories of the representation of categories in the observer.

For test examples that have a distance to the prototype, $d < d_-$, prototype theory and exemplar theory have distinct predictions. Take for example a test example that is identical to the prototype, $d = 0$. A prototype based observer should obviously be very likely to classify this example as a member of the category. The exemplar based observer would, however, be less likely to classify this example as a member of the category because the distance to the nearest example of a class member seen during training would be $d = d_- > 0$. According to the theories, this pattern should reverse when the distance to the prototype, d , approaches the distance to the prototype, d_- of the examples of class members seen during training. At this distance, the prototype-based observer would be less likely to identify the example as a member of the class because the distance to the prototype is $d = d_- > 0$ while the exemplar-based observer would be more likely to identify the example as a member of the class because the distance to the nearest example of a class member seen during training would, on average, be zero.

For test examples that have a distance to the prototype, $d > d_-$, exemplar and prototype theory alike predict that the chance of the observer categorising them as belong to the category should decrease as the distance increases. This is because both the distance to the prototype and the distance to the nearest example of a class member seen during training will increase when the distance to the prototype, d , increases. Hence, we cannot distinguish between the theories using the responses of the participants to these stimuli.

So far we have discussed the prediction of the theories with respect to the classification of test examples that are new, unseen during training. There is, however, another way to test the theories: presenting examples from the training phase in the test phase. For these test examples, the exemplar theory predicts that performance should be near perfect because a test example will be stored as an exemplar and therefore be recognised and classified correctly during testing. This should be the case for both members and non-members of the class. In contrast, according to prototype theory, the training examples would not be stored in memory. Only the prototype has been stored in memory. Therefore, the performance of the prototype based observer would be the same for test examples that were seen during training and for new, unseen, test examples if only their distance to the prototype is the same.

Human category learning exercise

In this exercise you will be the test participant in a prototype distortion task. The stimuli are configurations of three coloured disks. The location of the three disks determine whether the configuration belongs to a category, which we will call *leptons* only because this name have been used in other prototype distortion tasks described in the literature.

Instructions

- Start the experiment by running the Matlab script `PrototypeDistortionTask.m`
- The experiment starts with a training phase. At the beginning of the training phase you will be shown a window with a grid of displays. In each display there are three coloured disks. As the title of the window indicates, each display shows a member of the lepton class. Spend some time looking at the display and try to learn, in no specific way, what makes a lepton a lepton. Once you are done, click on the display. Now, another, similar, display appears. This display shows you examples of non-leptons. Spend some time looking at the display and try to get an idea of what lies outside of the lepton category. Click the display and the experiment will continue to the test phase.
- In the test phase you will be shown single displays of three coloured disks. Answer the question: "Is this a lepton?" by typing y for yes or n for no. Answer the question: "Is this a lepton?" by typing y for yes or n for no.
- When the test phase is over, the experiment ends and you will be shown two graphs of your results. The top graph shows the results for displays that you did *not* see during the training phase. The lower graph shows the results for displays that you *did* see during the training phase. For both

graphs, the distance to the prototype is on the x -axis. The y -axis shows you the proportion of responses in which you classified the display as a lepton.

- For this experiment, what are the predictions of the prototype and exemplar theories? Based on your results, did you behave as a prototype-based observer or a exemplar-based observer?

Bibliography

- Blakemore, C., & Cooper, G. F. (1970). Development of the brain depends on the visual environment. *Nature*, 228(5270), 477–478. <https://doi.org/10.1038/228477a0>
- Herlau, T., Mørup, M., & Schmidt, M. N. (2021). *Introduction to machine learning and data mining*. DTU lecture notes. <https://gitlab.compute.dtu.dk/tuhe/books>

